

Random-Access Ranked Retrieval and Similarity Search

Mohsen Dehghankar
University of Illinois Chicago
mdehgh2@uic.edu

Abolfazl Asudeh
University of Illinois Chicago
asudeh@uic.edu

Raghav Mittal
University of Texas at Arlington
rxm0006@mavs.uta.edu

Suraj Shetiya
IIT Bombay
surajs@cse.iitb.ac.in

Gautam Das
University of Texas at Arlington
gdas@uta.edu

Abstract

We extend *Random Access*, a fundamental operation that enables efficient search and exploration algorithms, to the modern interactive data systems based on *Ranked Retrieval* and *Similarity Search*, where orderings are dynamically defined over a *high-dimensional* feature space. This extension enables efficient solutions for a wide range of applications, from data analytics tools and database systems to recommendation systems and machine learning.

We formalize the *Random-Access Ranked Retrieval* (RAR) problem, and extend it to *Similarity Search*. Our algorithmic innovations include the development of a theoretically efficient algorithm based on geometric arrangements, achieving logarithmic query time. However, this method suffers from exponential space complexity in high dimensions. Therefore, we develop a second class of algorithms based on ϵ -sampling, which consume a linear space. Since exactly locating the tuple at a specific rank is challenging due to its connection to the range counting problem, we introduce a relaxed variant called κ -*Random-Access Ranked Retrieval*, which returns a small subset of size κ guaranteed to contain the target tuple. To solve this problem efficiently, we define an intermediate problem, *Stripe Range Retrieval* (SRR), and design a hierarchical sampling data structure tailored for narrow stripe range queries. Our method achieves practical scalability in both data size and dimensionality. We prove near-optimal bounds on the efficiency of our algorithms and validate their performance through extensive experiments on real and synthetic datasets, demonstrating scalability to millions of tuples and hundreds of dimensions.

1 Introduction

Random access to data is a foundational operation in computer science, enabling efficient algorithms for search, selection, and exploration. However, this operation largely breaks down in modern data systems where access is governed by ranking functions rather than static keys. In such systems, data items are ordered by a *query-specific*, *user-specific*, or *machine-generated* scoring function,¹ often *dynamically* defined over *high-dimensional* feature spaces. Yet, accessing the items at *arbitrary* rank positions or within a rank window typically requires fully materializing or sequentially scanning the ranked list, using operations such as *get-next*.

This limitation arises across a wide range of data-driven applications, including database systems, interactive data exploration and analytics tools, information retrieval and recommendation systems, and machine learning pipelines [13, 19, 21, 33, 42, 84]. Common scenarios include database queries on derived attributes [64, 68],

analytical exploration for business decisions [68], paged navigation of search results [33], browsing ranked product lists in e-commerce [78], and scrolling through personalized feeds on social platforms [65].

A concrete example arises in web application APIs, which implement this through paged or offset-based navigation:

EXAMPLE 1. *In web and e-commerce applications, APIs commonly use offset-based pagination. For instance, a request such as [GET /items?offset=400&limit=5] retrieves items ranked 401–405. More generally, this pattern requires efficient retrieval of results at deeper ranks without scanning all earlier entries [78, 90].*

Similar scenarios also arise in data science and visual analytics tooling, where analysts frequently interact with large, ranked datasets. Tools such as spreadsheets, notebooks, and visual analytics platforms provide intuitive interfaces for exploration [30, 82, 86, 100], but the explosive growth of data volumes driven by big data and AI has placed heavy demands on their efficiency and responsiveness [17]. For example, while using data science workflows such as Jupyter notebooks [62], analysts may need random access to particular ranks or quantiles (e.g., 10th, 20th, etc.) in a sorted dataframe, effectively *skipping* earlier-ranked items.

Beyond ranked retrieval, closely related challenges arise in similarity search, where items are ordered by their distance or similarity to a query object.

EXAMPLE 2. *In similarity search applications, such as nearest-neighbor queries over high-dimensional embeddings, items are ranked by their similarity to a query point. While existing systems efficiently retrieve the top- k most similar items, users and downstream applications may need to access items at arbitrary similarity ranks or within specific distance bands. For example, an analyst may wish to examine the points whose similarity lies within a narrow range around a decision boundary.*

Traditionally, research on ranked retrieval has focused on *top- k* query processing, where the goal is to identify the highest-ranked tuples [34, 44, 53, 54]. In this setting, results are retrieved sequentially from the top of the ranking up to position k , where k is typically a small constant relative to the dataset size. Similarly, in the context of similarity search, *k*-nearest neighbor (kNN) search aims to retrieve the k most similar points to a query [70, 79, 97]. These approaches are designed for small values of k and, as shown in our evaluations, become ineffective for random-access queries that seek to retrieve a tuple at an arbitrary rank position i , where i may be as large as $O(n)$.

¹Also known as ranking functions.

Direct access to conjunctive queries in database systems has been studied to directly access a specific position in the join result, without fully materializing the join [14, 23, 42, 93]. However, these works assume a *prespecified ordering* of the tuples (e.g., lexicographic order on an attribute) and focus on skipping the join operation. As a result, such approaches are not suitable for settings where the ranking function is part of the input query.²

Contributions. In this paper, we formalize and study the *Random-Access Ranked Retrieval* (RAR) problem and the *Random-Access Similarity Search* problem (RAS) (Section 3). We show a reduction from RAS to RAR problem. Next, we propose an algorithm for RAR that uses computational geometric concepts of duality, arrangements, and levels of arrangements. However, this algorithm suffers from the curse of dimensionality, and its space complexity increases exponentially with dimension (Appendix E).

Consequently, we propose several algorithms with linear space complexity even for a large number of attributes, based on the concept of ϵ -samples (Section 6). We start from the two-dimensional case and extend our algorithm to higher dimensions by combining it with the stripe range searching problem. This approach offers an efficient solution for finding a conformal set for an arbitrary rank position. Efficiently finding the tuples within a *narrow stripe range* is fundamental in the development of our solution. We formulate this as the *stripe range retrieval* (SRR) problem, focusing on developing efficient algorithms for the challenging narrow ranges that contain only a few tuples.

Inspired by the Hierarchical Navigable Small World graphs for approximate nearest neighbor search [37, 70], our practical and scalable algorithm constructs a hierarchical sampling structure for answering SRR, maintaining the neighborhood regions of points in the hierarchy using smallest enclosing balls (Section 6.3).

Theoretically, we show that our algorithms achieve near-optimal efficiency up to a logarithmic factor (Appendix L). In addition, we conduct extensive experiments on real-world and synthetic datasets to evaluate the performance of our algorithms in practice and compare them with existing baselines (Section 7). Our experiments confirm the *efficiency and scalability* of our proposed algorithms. In particular, the ϵ -sampling algorithm, when combined with the hierarchical structure, achieves the best scalability with respect to both dimension and dataset size in terms of query time and index size, whereas the baseline methods are mostly affected by the curse of dimensionality.

Due to space limitations, we defer extended discussions on applications, related work, more general settings (including dynamic and non-linear scoring functions), additional experiments, and omitted proofs and pseudocode to the appendix.

2 Applications Overview

Random-access ranked retrieval (RAR) arises naturally across a broad range of modern data systems that rely on query-dependent ranking functions. While Examples 1 and 2 illustrate its relevance in web and e-commerce applications, similar access patterns appear in databases, data analytics platforms, and decision-support systems. Below, we briefly summarize several representative application domains; a detailed discussion with concrete examples is provided in Appendix A. Together, these applications highlight RAR as a

general-purpose access primitive with broad impact across ranked data management and analysis.

Extending Random Access to Ranked Retrieval. Random access to arbitrary positions in arrays and sorted lists is a fundamental algorithmic primitive, enabling classic techniques such as **binary search**. In contrast, ranked retrieval systems typically only support sequential access, where retrieving the item at rank i requires enumerating all higher-ranked results. RAR bridges this gap by enabling direct access to arbitrary rank positions under a ranking function, thereby extending a broad class of random-access-based algorithms to query-dependent, high-dimensional ranking settings.

Analytical Insights and Decision Support. Many analytics and business decision-making tasks require understanding how key metrics behave across different rank positions or percentiles, rather than focusing solely on top- k results. Examples include boundary analysis in targeted advertising, percentile-based customer analytics, and sensitivity analysis around decision cut-offs [32, 48, 89, 101]. Such tasks require efficient access to arbitrary quantiles and narrow rank intervals, which RAR supports without exhaustive scanning.

Database Queries over Derived Attributes. Modern database and spreadsheet systems frequently operate over *derived attributes* computed at query time, such as weighted combinations or transformations of existing columns [46, 64, 68, 83]. Since these attributes do not exist at indexing time, supporting efficient ranked access over them is challenging. By treating derived attributes as query-defined scoring functions, RAR enables scalable access to arbitrary rank positions and rank ranges without materializing or sorting the full result set.

Individually Fair Decision Boundaries. In score-based decision systems, cut-off points often serve as decision boundaries and must be chosen carefully to ensure interpretability and individual fairness [6, 15, 66, 74]. Identifying meaningful boundaries requires localized inspection of ranked items around candidate cut-off points to detect significant score gaps. RAR enables such narrow, rank-local exploration, supporting principled and responsible boundary selection.

3 Formal Definitions

We consider a dataset $\mathcal{D} = \{p_1, p_2, \dots, p_n\}$ consisting of n points in $\mathbb{R}^d$, where each point $p \in \mathcal{D}$ has d real-valued (scoring) attributes, aka. features, used for ranking.³ A *scoring function* $\text{score} : \mathbb{R}^d \rightarrow \mathbb{R}$ assigns a real-valued score to each point in the dataset.⁴ We consider the *linear scoring functions*, where the score is defined as a weighted sum of the features. Specifically, given a weight vector $f \in \mathbb{R}^d$, the score assigned to a point p is computed as the dot product⁵: $\text{score}_f(p) = f^\top p = \sum_{j=1}^d f[j]p[j]$. We may directly call f as the scoring function. Figure 1 (left) provides an example of a dataset $\mathcal{D}$ with $n = 16$ points and two attributes x and y . Let the red origin-anchored ray in the figure represent f . The perpendicular projection of each point p on the vector f shows $\text{score}_f(p)$.

Our work extends beyond linear functions, supporting (a) scores expressed as linear combinations of non-linear terms (Appendix C)

²Please refer to Appendix B for a complete review of the related work.

³Besides scoring attributes, the dataset may contain other attributes such as non-ordinal descriptive attributes that are not used for ranking. In this paper, we use the terms attribute and feature interchangeably to refer to the scoring attributes.

⁴We use the terms scoring function and ranking function interchangeably.

⁵We assume that the scoring vector f is normalized, i.e., $\|f\| = 1$.

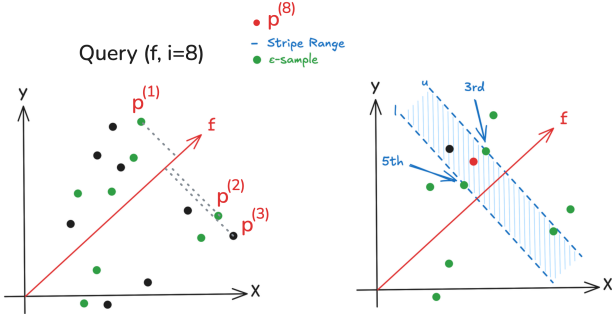


Figure 1: A visual representation of Eps2D algorithm. The toy dataset contain 16 points and the query asks for $(i = 8)$ -th point with the scoring function shown in red. (Left) First, we find the bounds by solving RAR on the ϵ -sample. (Right) Finding the ℓ and u values, we solve the SRR problem on the stripe range.

and (b) common similarity measures, like Euclidean and Cosine similarity (Section 3.3).

3.1 Random-Access Ranked Retrieval (RAR)

Given a scoring function f , we define a total order over the dataset $\mathcal{D}$ by sorting the points in descending order of their scores. For example, in Figure 1, the ordering of the projections on the vector of f shows their ranking. Let $\text{rank}_{\mathcal{D},f} : \mathcal{D} \rightarrow \mathbb{N}$ denote the function that assigns to each point in $\mathcal{D}$ its rank in this ordering (i.e., $\text{rank}_{\mathcal{D},f}(p) = i$ if p is the i -th highest-scoring point under score_f). When the dataset $\mathcal{D}$ is clear from the context, we write rank_f for brevity. We denote by $p^{(i)}$ the i -th point (aka the i -th element) under score_f , that is, $p^{(i)} = \text{rank}_f^{-1}(i)$, $\forall 1 \leq i \leq n$. The main problem is to efficiently find the point at a specific rank i .

PROBLEM 1 (κ -RANDOM-ACCESS RANKED RETRIEVAL (κ -RAR)). Given a dataset $\mathcal{D}$ available at preprocessing time, a query consists of a linear scoring function $f \in \mathbb{R}^d$ and an integer $i \in [n]$, find a conformal set $C(i) \subset \mathcal{D}$, where $|C(i)| = \kappa$ and $p^{(i)} \in C(i)$.

Based on this definition, when $\kappa = 1$, the task reduces to identifying the exact i -th ranked element. As shown in Appendix L, finding $p^{(i)}$ in high-dimensional spaces is challenging, as its complexity is tied to that of the *half-space range counting* problem [2, 26]. For $1 < \kappa \ll n$, this formulation naturally leads to a relaxed variant of the problem: instead of returning a single point, the output is a small set that is guaranteed to contain $p^{(i)}$. Inspired by the notion of conformal prediction in machine learning [8, 85], we refer to this set as the *conformal set* for rank i .⁶

3.2 Stripe Range Retrieval (SRR)

While addressing Problem 1, we also solve a side problem that finds all points that fall in a stripe range. Formally, a *stripe range* $\mathcal{S}_{f,\ell,u}$ is defined by a scoring function $f \in \mathbb{R}^d$ and two real-valued boundaries $\ell, u \in \mathbb{R}$ such that $\ell \leq u$. It corresponds to the universe of all points $x \in \mathbb{R}^d$ whose scores based on f lies within the interval $[\ell, u]$, i.e.,

$$\mathcal{S}_{f,\ell,u} = \{x \in \mathbb{R}^d \mid \ell \leq \text{score}_f(x) \leq u\}.$$

⁶We remove κ when it is clear from the context or when we refer to the general problem for any κ .

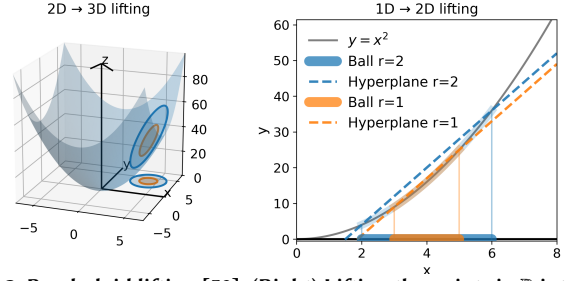


Figure 2: Paraboloid lifting [50]. (Right) Lifting the points in $\mathbb{R}$ into $\mathbb{R}^2$, which transforms segments (1D Balls) into the intersection of half-planes and parabola ($y = x^2$). (Left) Similarly, the $\mathbb{R}^2$ to $\mathbb{R}^3$ lifting.

Given a stripe range query, we aim to return the set of points in $\mathcal{D}$ that fall inside the given stripe range. For example, in Figure 1, the stripe range query is highlighted between the two blue lines.

PROBLEM 2 (STRIPE RANGE RETRIEVAL (SRR)). Given a dataset $\mathcal{D}$ and a query consisting of a stripe range $\mathcal{S}_{f,\ell,u}$, return all points in $\mathcal{D}$ that lie within this range, i.e., $\mathcal{D}_0 = \mathcal{S}_{f,\ell,u} \cap \mathcal{D}$.

3.3 Random-Access Similarity Search (RAS)

Similar to the ranked retrieval problem, we define a *random-access variant of the similarity search problem* (also known as nearest neighbor search) under a given similarity metric.

Formally, let $\mathbf{d} : \mathcal{D} \times \mathcal{D} \rightarrow \mathbb{R}$ be a distance function that induces the similarity metric on the dataset $\mathcal{D}$. For a query point q , we define a ranking function $\text{nn_rank}_q : \mathcal{D} \rightarrow \mathbb{N}$, which orders points in $\mathcal{D}$ by decreasing similarity to q (equivalently, by increasing distance $\mathbf{d}(q, \cdot)$). In other words, the most similar point to q (i.e., the nearest neighbor) has rank 1.

Despite the classical NN problem that retrieves the k -nearest neighbors for a constant k , we consider a more general *random-access* problem, retrieving the point in rank i for any $i \in [n]$.

PROBLEM 3 (κ -RANDOM-ACCESS SIMILARITY SEARCH (κ -RAS)). Given a dataset $\mathcal{D}$ and a similarity metric $\mathbf{d}$ available at preprocessing time, a query consists of a point $q \in \mathbb{R}^d$, and an integer $i \in [n]$, find a conformal set $C(i) \subset \mathcal{D}$, where $|C(i)| = \kappa$ and $p^{(i)} = \text{nn_rank}_q^{-1}(i) \in C(i)$.

Similar to Problem 1, when $\kappa = 1$, we look for the exact i -th point, while for $1 < \kappa \ll n$, we have a relaxed version of the problem.⁶

In this work, we focus on two widely used distance metrics in ANN search and practical similarity search systems: the ℓ_2 -norm (Euclidean distance) and cosine similarity.

In the following, we first show how the RAS problem can be reduced to RAR. Subsequently, we focus on solving RAR, which represents the more general problem.

3.4 Reducing RAS to RAR

In this section, we use existing techniques to reduce the RAS problem to the RAR problem, and show that an efficient solution to the RAR problem would also solve the RAS problem.

Euclidean distance: Given a query point $q \in \mathbb{R}^d$, the ℓ_2 -norm similarity of a point $p \in \mathcal{D}$ to q is measured as $\mathbf{d}(p, q) = \|p - q\|_2$. In Appendix D.1, we prove Lemma 1, which provides the reduction of the RAS problem to the RAR problem for ℓ_2 -norm:

LEMMA 1. Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d+1}$ denote the paraboloid lifting transform [50], which maps a point in $\mathbb{R}^d$ to $\mathbb{R}^{d+1}$ (see Figure 2). The transformation is defined as

$$\phi(p) = (p[1], p[2], \dots, p[d], \|p\|_2^2).$$

Using this mapping, Euclidean balls in $\mathbb{R}^d$ correspond to halfspaces in $\mathbb{R}^{d+1}$. Moreover, using the scoring function

$$f = (-2q[1], -2q[2], \dots, -2q[d], 1),$$

the RAS problem on query q reduces to the RAR problem on f .

Cosine distance: Cosine distance is a dissimilarity measure derived from cosine similarity (more specifically, $1 - \text{cosine similarity}$) that captures the angular separation between two vectors. For a query q , the cosine distance is given by $\mathbf{d}(p, q) = 1 - (q^\top p) / (\|q\|_2 \cdot \|p\|_2)$. Lemma 2, proven in Appendix D.2, provides the reduction of the RAS problem to the RAR problem for this distance:

LEMMA 2. Let $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^d$ denote the normalization map, which scales each point to unit ℓ_2 norm. The transformation is defined as

$$\phi(p) = \frac{p}{\|p\|_2}.$$

Using this mapping, the RAS problem with query point q , can be reduced to RAR problem with the scoring function:

$$f = \left(\frac{q[1]}{\|q\|_2}, \dots, \frac{q[d]}{\|q\|_2} \right).$$

Note that these reductions equally apply to any value of $\kappa \geq 1$. These transformations, map a ball in κ -RAS problem to a half-space in κ -RAR problem (see Figure 2). As a result, from now on, we focus only on the general κ -RAR problem.

4 Background

In this section, we summarize the key concepts and definitions used in this paper. We assume to be given a range space $(\mathcal{D}, \mathcal{R})$, where $\mathcal{D}$ is the dataset, and $\mathcal{R}$ is a collection of geometric ranges defined over $\mathcal{D}$, such as all possible balls, half-spaces, or other ranges.

VC-Dimension. The Vapnik-Chervonenkis (VC) dimension [95] for a range space is the size of the largest subset $C \subseteq \mathcal{D}$ such that every subset of C can be realized as the intersection of C with some range in $\mathcal{R}$. A low VC-dimension implies a limited expressive power, which in turn allows for efficient sampling-based approximations [50].

ϵ -Samples. An ϵ -sample of a range space $(\mathcal{D}, \mathcal{R})$ is a subset $\mathcal{N} \subseteq \mathcal{D}$ such that, for every range $R \in \mathcal{R}$, the proportion of points in R is approximately preserved in $\mathcal{N}$. More formally, $\mathcal{N}$ is an ϵ -sample if for all $R \in \mathcal{R}$,

$$\left| \frac{|R \cap \mathcal{D}|}{|\mathcal{D}|} - \frac{|R \cap \mathcal{N}|}{|\mathcal{N}|} \right| \leq \epsilon.$$

For more details on ϵ -sampling, we refer the reader to [50]. We use the following foundational theorem on ϵ -samples as a key tool in deriving our theoretical guarantees:

THEOREM 3 (ϵ -SAMPLE THEOREM [95]). Let $(\mathcal{D}, \mathcal{R})$ be a range space of VC-dimension δ . Then for any $\epsilon, \phi \in (0, 1)$, a random sample $S \subseteq \mathcal{D}$ of size $O\left(\frac{\delta}{\epsilon^2} \log \frac{\delta}{\epsilon} + \frac{\delta}{\epsilon^2} \log \frac{\delta}{\phi}\right)$ is an ϵ -sample of $(\mathcal{D}, \mathcal{R})$ with probability at least $1 - \phi$. $\square$

In addition to random sampling, which provides a probabilistic method for constructing ϵ -samples, there also exist deterministic construction algorithms based on discrepancy theory [27, 50].

5 Solution Overview

In this section, we provide a high-level overview of our proposed algorithms and summarize their theoretical guarantees.

The baseline approach for answering the RAR queries uses the extension of the classic divide and conquer (D&C) median finding algorithm, which, instead of the median, finds the element at position i [18, 31]. To do so, it first makes a linear pass over $\mathcal{D}$, and for each point p , computes $\text{score}_f(p)$. Then, using the D&C algorithm, it finds $\text{rank}_f^{-1}(i)$ in $O(n)$. The Quick Select algorithm⁸ also provides a randomized version of this baseline. Instead, our algorithms, summarized in Table 1, provide more efficient solutions by preprocessing the data and constructing proper data structures:

KTHLEVEL. Our first algorithm for solving the RAR problem uses the computational geometric concepts of *duality* and *arrangement of hyperplanes* [29, 41]. This algorithm, named KTHLEVEL, keeps track of different *levels of the arrangement*, which are searched at query time to efficiently answer RAR queries, i.e., finding the tuple at a given rank position i . Due to the space constraints, further details about this algorithm and its analysis are presented in Appendix E.

This method is an exact solution to the 1-RAR problem ($\kappa = 1$) that offers highly efficient *logarithmic query time*, but it is not practical for high-dimensional settings ($d > 2$) as it has a *space complexity exponential to the number of attributes d* . Our next algorithms based on ϵ -samples aim to maintain linear space complexity.

ϵ -sampling. We start with the simple two-dimensional case, also illustrated in Figure 1. During the preprocessing, we compute an ϵ -sample of the dataset, denoted by $\mathcal{N}_\epsilon$. Given a scoring function f and a target rank i , we approximately solve the RAR problem on the smaller set $\mathcal{N}_\epsilon$, rather than the full dataset $\mathcal{D}$. From this approximate solution, we derive a stripe range containing the true i -th ranked point. By solving an instance of stripe range searching (SRR) we find this exact point. We refer to this algorithm as Eps2D. A visual representation of this approach is shown in Figure 1.

For higher-dimensional settings, we follow a similar generalized strategy (EpsRANGE). However, unlike in 2D, standard range searching algorithms do not perform well in high-dimensional datasets. To overcome this, we propose a practical algorithm to solve the intermediate SRR problem with hierarchical sampling (EpsHIER).

Lower Bounds and Optimality. A discussion on the lower bounds and optimality of our algorithms is provided in the Appendix L.

6 ϵ -sampling Approach

In theory, the solution based on constructing the arrangement of dual hyperplanes provides a logarithmic time for answering the RAR queries (see Appendix E). However, it is not practical, especially when the number of ranking attributes is not small ($d \geq 3$), since the size of the arrangement (hence, the space complexity) increases exponentially with d . Therefore, in this section, we propose practical solutions that keep the space complexity linear, i.e., $O(n)$. Our algorithms are based on the computational geometric concept of ϵ -samples [50] (also introduced in Section 4).

In the following, We start by discussing the two-dimensional Eps2D algorithm, then we will generalize our solution to the higher

⁸<https://en.wikipedia.org/wiki/Quickselect>

Algorithm Name	Summary	Dimension	Space Complexity	Query Time	Problem
KTHLEVEL	Following the levels of arrangement	$d \geq 2$	$O(n^d)$	$O(\log n)$	1-RAR
Eps2D	ϵ -sampling in 2D	$d = 2$	$O(n)$	$O(n^{2/3} \log n)$	1-RAR
EpsRANGE	ϵ -sampling + Range Searching	$d \geq 2$	$O(n)$	$O(\max\{n^{1-1/d}, \frac{1}{\epsilon^2} \log \frac{1}{\epsilon}\})$ $O(\max\{n^{1-1/d}, n^{2/3} \log n\})$	(ϵn) -RAR 1-RAR
EpsHIER	ϵ -sampling + Hierarchical Sampling	$d \geq 2$	$O(n)$	Worst-case $O(n)$; Practically Efficient	(ϵn) -RAR

Table 1: Overview of the proposed methods for the RAR problem.⁷

dimensions, and propose practical methods to solve the intermediate SRR problem (Problem 2).

6.1 Eps2D: ϵ -sampling in 2D

In this section, we discuss our solution based on ϵ -samples in 2D, called Eps2D, also illustrated in Figure 1.

Preprocessing. During the preprocessing phase, the algorithm computes an ϵ -sample of the dataset $\mathcal{D}$, where the ranges are *stripe ranges*. We denote this ϵ -sample by $\mathcal{N}_\epsilon$, for a given parameter ϵ .

Query Answering. Let $m = |\mathcal{N}_\epsilon|$ denote the size of the ϵ -sample built during the preprocessing, where $m \ll n$.⁹ We take the following steps to answer the query:

(Step i: *Thresholding*) During the *query phase*, given a query pair (f, i) , we begin by solving the 1-RAR problem on the ϵ -sample $\mathcal{N}_\epsilon$. Specifically, we identify the i_ℓ -th and i_u -th ranked points in $\mathcal{N}_\epsilon$ with respect to the scoring function f , where:

$$i_\ell = \left\lceil m \left(\frac{i}{n} - \epsilon \right) \right\rceil, \quad i_u = \left\lceil m \left(\frac{i}{n} + \epsilon \right) \right\rceil, \quad (1)$$

and $m = |\mathcal{N}_\epsilon|$. This step can be done by sorting the points in $\mathcal{N}_\epsilon$ according to score_f , or by using linear-time-in- m baseline median-finding algorithms, such as median-of-medians. These two points serve as score-based thresholds that are guaranteed to bound the score of the i -th ranked point in the full dataset $\mathcal{D}$, as stated in the following lemma.

LEMMA 4. *Let $\mathcal{N}_\epsilon$ be an ϵ -sample of the dataset $\mathcal{D}$ for some $\epsilon > 0$. Then, for any linear scoring function f , the score of the i -th ranked point $p^{(i)}$ in $\mathcal{D}$ is bounded as*

$$\text{score}_f(q_\ell) \leq \text{score}_f(p^{(i)}) \leq \text{score}_f(q_u),$$

where q_ℓ and q_u are the i_ℓ -th and i_u -th ranked points in $\mathcal{N}_\epsilon$ under f , i.e.,

$$q_j = \text{rank}_{\mathcal{N}_\epsilon, f}^{-1}(i_j), \quad \text{for } j \in \{\ell, u\}.$$

PROOF. See Appendix G for proof. $\square$

(Step ii: *SRR*) Next, we compute the score thresholds corresponding to the boundary points identified in the previous step:

$$\ell = \text{score}_f(q_\ell), \quad u = \text{score}_f(q_u). \quad (2)$$

Using these values, we define the stripe range $\mathcal{S}_{f, \ell, u}$. We then solve an instance of the SRR problem on the dataset $\mathcal{D}$ using this stripe range. The result is a subset of points within the score interval $[\ell, u]$, which we denote by $\mathcal{D}_o$. This set contains the i -th ranked point $p^{(i)}$, since $\mathcal{N}_\epsilon$ is an ϵ -sample. Note that this set is a

conformal set for rank i , hence providing a solution to the relaxed $|\mathcal{D}_o|$ -RAR problem.

(Step iii: *Range Counting*) We define the half-space H_u as,

$$H_u = \{p \in \mathcal{D} \mid p^\top f \geq u\}, \quad (3)$$

H_u contains all points in $\mathcal{D}$ whose score under f is at least u . We then apply a half-space range counting algorithm in two dimensions to compute the number of points lying within H_u [71].

(Step iv: *Final Selection*) In the final step, we sort the points in the candidate set $\mathcal{D}_o$ in descending order according to the scoring function score_f . Let $|H_u|$ denote the number of points in $\mathcal{D}$ whose score is greater than or equal to u , as computed in the previous step. We then return the $(i - |H_u|)$ -th point in the sorted list as the exact answer to the RAR query.

A pseudo-code of Eps2D is shown in Algorithm 2 in Appendix K. The analysis of the space and time complexity of Eps2D algorithm is also provided in Appendix F. The result can be summarized in the following theorem:

THEOREM 5. *The Eps2D algorithm solves the exact RAR problem in 2D using linear space in n and $O(n^{2/3} \log n)$ query time.*¹⁰

6.2 EpsRANGE: ϵ -sampling in Higher Dimensions

Now, we present the generalization of the Eps2D algorithm to higher dimensions ($d \geq 2$), referred to as the EpsRANGE algorithm. The preprocessing phase remains unchanged: we compute an ϵ -sample $\mathcal{N}_\epsilon$ of the dataset $\mathcal{D}$, of size m , for a chosen parameter ϵ . We know that $m \ll n$, because the VC-dim of stripe ranges is $\delta = d + 1$ [50].

During query time, the algorithm proceeds similarly to Eps2D. In Step i, we find the i_ℓ -th and i_u -th points in $\mathcal{N}_\epsilon$ according to the scoring function f . In Step ii, we solve a high-dimensional instance of the SRR problem. The resulting set $\mathcal{D}_o$ contains candidate points from $\mathcal{D}$ that lie between the scores of the i_ℓ -th and i_u -th ranked points in the sample. This set is a conformal set for the relaxed problem $|\mathcal{D}_o|$ -RAR. To obtain the exact solution, we proceed to Step iii, where we perform a high-dimensional range counting query to determine the exact rank of each point in $\mathcal{D}_o$. Finally, in Step iv, we sort the candidates in $\mathcal{D}_o$ and return the point with exact rank i , completing the solution to the RAR selection problem.

Analysis. The analysis is similar to the 2D case, provided in Appendix I. The results are as follows:

THEOREM 6. *The EpsRANGE algorithm solves the κ -RAR problem with linear space usage and $O(\max\{n^{1-1/d}, \frac{d}{\epsilon^2} \log \frac{d}{\epsilon}\})$ time, where $\kappa = O(\epsilon n)$. The exact version, 1-RAR, is also solved with the same space usage and $O(\max\{n^{1-1/d}, dn^{2/3} \log n\})$ time.*

⁸The Big-Oh notations in our guarantees hide constants that are polynomially dependent on the dimension d . Consistent with the prior literature [2], our focus is on the exponential dependence on d .

⁹This comes from Theorem 3, while noting that the VC-dimension of stripe ranges is $\delta = d + 1$.

¹⁰Note that we assumed the set $\mathcal{N}$ to be an ϵ -sample. According to Theorem 3, this happens with high probability via random sampling.

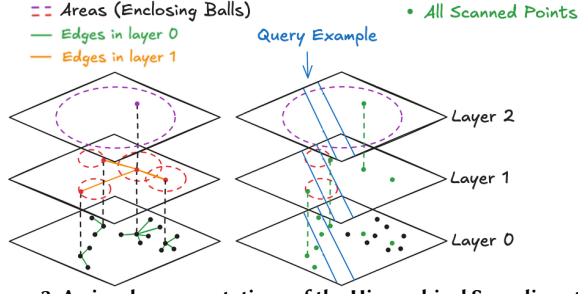


Figure 3: A visual representations of the Hierarchical Sampling structure for solving SRR problem. In this setting, decay rate $r = 4$. (Left) The data structure built during the preprocessing. (Right) An example of running a query on top of this structure, the green points are all the points scanned during the query phase.

6.3 Stripe Range Searching with Hierarchical Sampling

The main bottleneck in the runtime of the EPSRANGE algorithm lies in the range searching subroutines used to solve the SRR problem. These subroutines often suffer from the curse of dimensionality, particularly for general simplices, such as non-orthogonal half-spaces within stripe ranges. Even algorithms with theoretical sublinear time guarantees tend to perform poorly in practice [72], with a runtime comparable to or worse than a naive linear scan over the entire dataset (see experiments in Section 7).

To address this limitation, we propose a practical algorithm for solving the SRR problem, where the ranges are stripes in d dimensions. We empirically show that augmenting EPSRANGE with this solution leads to significant performance improvements on real-world high-dimensional datasets (EPSHIER algorithm).

We present a hierarchical data structure for solving the stripe range searching problem. We begin with a high-level overview of the structure, followed by a description of the *preprocessing step*, and conclude with the details of the *query phase* of the algorithm.

Overview. We observe that the stripe range queries constructed in the solution of the RAR problem correspond to *narrow regions*, especially for small choices of ϵ . To exploit this, we construct a hierarchical structure over the input point set that enables efficient pruning of irrelevant points when answering such range queries.

The core idea is to organize the points in a hierarchy that preserves spatial proximity, inspired by similar structures used in nearest neighbor search [37, 51, 70, 77].¹¹ Each point in the hierarchy maintains a list of neighbors, and during preprocessing, we compute and store an *enclosing ball* around each such neighborhood.¹²

During query processing, we employ a simple yet effective heuristic: if the stripe intersects the enclosing ball of a node, we explore the node; otherwise, we safely prune it. This significantly reduces the number of points examined during query time.

Empirical results demonstrate that for the narrow stripe ranges used in the RAR problem, this approach leads to a fast algorithm by eliminating many unnecessary explorations. See Figure 3 (right), which illustrates the points explored during an example query.

6.3.1 Preprocessing.

¹¹See Related Work (Appendix B) for comparison.

¹²Not necessarily centered at that point.

Layers. Given the input dataset $\mathcal{D}$, we construct a hierarchical graph $\mathcal{G}(\mathcal{D})$ on top of it. The graph consists of L layers built via recursive random sampling. We define the base layer as the dataset itself, $\mathcal{L}_0 = \mathcal{D}$. Each subsequent layer $\mathcal{L}_\ell$ is formed by randomly sampling $\frac{|\mathcal{L}_{\ell-1}|}{r}$ points from the previous layer $\mathcal{L}_{\ell-1}$, where r is a hyperparameter known as the *exponential decay rate*. Thus, the size of each layer satisfies:

$$|\mathcal{L}_\ell| = \frac{|\mathcal{L}_{\ell-1}|}{r}, \quad \forall 1 \leq \ell \leq L$$

Edges. Once the layer $\mathcal{L}_\ell$ is sampled, each point in $\mathcal{L}_\ell$ serves as a centroid for partitioning the points in the layer below, $\mathcal{L}_{\ell-1}$. Specifically, each point $p \in \mathcal{L}_{\ell-1}$ is connected to its nearest neighbor in $\mathcal{L}_\ell$, forming a *directed edge* from the centroid c^* to p , where

$$c^* = \arg \min_{c \in \mathcal{L}_\ell} \|p - c\|_2. \quad (4)$$

This procedure creates a layered structure, where each level induces a graph by connecting lower-layer points to their nearest centroids in the current layer (see Figure 3, left, where each level illustrates the induced graph). For every node p in the graph $\mathcal{G}(\mathcal{D})$, let $N_\ell(p)$ denote the set of its neighbors at layer $\mathcal{L}_\ell$. For example, in Figure 3 (left), the root node has three red neighbors at layer 1 and five black neighbors at layer 0.

Area of Nodes. For each node p in layer ℓ of the graph $\mathcal{G}(\mathcal{D})$, we associate a set called the *area* of the node, denoted by $\mathcal{A}_\ell(p)$. The area captures the set of points from the base layer that are hierarchically covered by this node. The definition is recursive. For the base layer, we set: $\mathcal{A}_0(p) = \{p\}$. For any node p in layer $\ell > 0$, the area is defined as:

$$\mathcal{A}_\ell(p) = \bigcup_{q \in N_{\ell-1}(p)} \mathcal{A}_{\ell-1}(q),$$

where $N_{\ell-1}(p)$ denotes the neighbors (i.e., children) of p in layer $\ell - 1$. In other words, the area of a node at layer ℓ is the union of the areas of all its children in the layer below. As illustrated in Figure 3, we represent the area of each node using the smallest enclosing circle, shown with dotted lines.

Enclosing Balls. For each node p at layer ℓ , we define $\mathcal{B}_\ell(p)$ as the *smallest enclosing ball* that covers the area $\mathcal{A}_\ell(p)$. These balls are later used during query time to enable efficient pruning of nodes when searching within a stripe range:

$$\mathcal{B}_\ell(p) = \text{Enclosed-Ball}(\mathcal{A}_\ell(p)).$$

Construction Algorithm. To construct the hierarchical graph $\mathcal{G}(\mathcal{D})$, we employ a bottom-up recursive approach. Starting from the base layer, we iteratively sample a subset of points to form the next layer. At each level, we connect nodes in the current layer to their nearest centroids in the layer above, thereby forming directed edges. After establishing the connections, we compute and update the area sets and corresponding enclosing balls for the newly constructed layer. The pseudo-code of the preprocessing procedure is detailed in Algorithm 3 in Appendix K. Please refer to the Appendix J for the analysis of this algorithm.

Querying Algorithm. Given a stripe range $\mathcal{S}_{f,\ell,u}$, the goal is to report all points in $\mathcal{S}_{f,\ell,u} \cap \mathcal{D}$. The query process begins at the top of the hierarchy, starting from layer $\mathcal{L}_L$. At each layer ℓ , we examine whether the enclosing ball $\mathcal{B}_\ell(p)$ of each point $p \in \mathcal{L}_\ell$ intersects the stripe $\mathcal{S}_{f,\ell,u}$. If there is no intersection, the corresponding node is pruned from further exploration. Otherwise, we proceed to explore

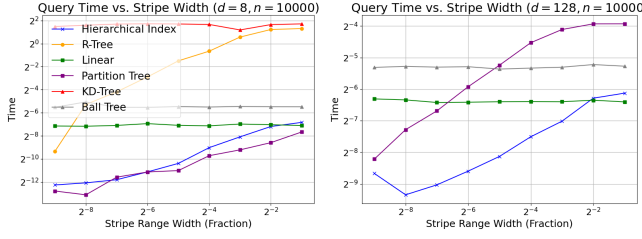


Figure 4: Comparison of SRR query time with respect to the stripe width across different dimensionalities on the synthetic data.

its neighbors in the next lower layer, denoted by $N_{l-1}(p)$. The full query procedure is provided in Algorithm 4 in Appendix K. A visual illustration of this process is shown in Figure 3 (right).

7 Experiments

In this section, we evaluate the performance of our algorithms in both the random-access ranked retrieval and similarity search settings. Our implementation is available at [this anonymous repository](#). We begin by describing the datasets. Next, we present results for the *stripe range searching* subtask showcasing our proposed hierarchical index (Subsection 7.1). We then evaluate our methods on two primary tasks: *random-access ranked retrieval* (Subsection 7.2) and *random-access similarity search* (Subsection 7.3). Additional results and experimental details are deferred to Appendix M.

Datasets. For the ranked retrieval, we use synthetic and real-world datasets. The synthetic dataset is generated from a Zipfian distribution. Our real-world datasets include US USED CARS [75] (3M records, 66 columns), FIFA 2023 [63] (300K records, 54 columns), and US FLIGHTS [94] (5M records, 31 columns). For the similarity search setting, we evaluate our methods on five popular benchmark datasets from [ann-benchmarks.com](#): SIFT-128-EUCLIDEAN [58] (1M vectors, ℓ_2), FASHION-MNIST-784-EUCLIDEAN [99] (60K vectors, ℓ_2), GIST-960-EUCLIDEAN [58] (1M vectors, ℓ_2), GLOVE-100-ANGULAR [80] (1M vectors, cosine), and GLOVE-25-ANGULAR [80] (1M vectors, cosine).¹³ More details of the datasets are provided in Appendix M.1.

7.1 Stripe Range Retrieval

For the SRR problem, we compare against classical and widely used range searching indices, including KD-Tree [16], R-Tree [49], and Partition Tree [71], and Ball Trees [77]. Our proposed hierarchical index is abbreviated as Hierarchical. As a baseline, we also include a simple linear search approach, denoted as Exhaustive. To apply the baseline range searching indices to stripe range queries, we traverse the tree top-down, pruning subtrees whose corresponding hyperrectangles do not intersect the query stripe (see Algorithm 4). More experiments on SRR problem is provided in Appendix M.2.

7.1.1 Synthetic Dataset. We begin by evaluating the performance of the algorithms on synthetic datasets.

Effect of Stripe Width. For these experiments, the scoring function f is sampled uniformly from the unit hypersphere in $\mathbb{R}^d$. We vary the stripe $\mathcal{S}_{f,\ell,u}$ by adjusting its width, i.e., the number of points it contains. Figure 4 presents the query time performance across different dimensionalities d as a function of stripe width. We

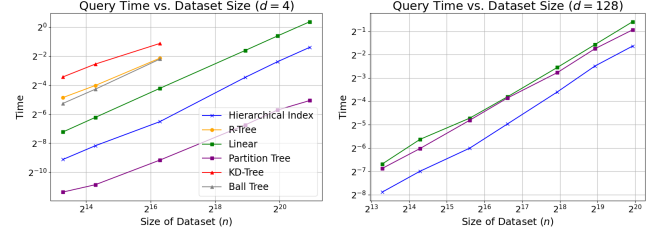


Figure 5: Comparison of the query time of Hierarchical Sampling on SRR problem vs dataset size. Large d and n cases do not use KD-tree or R-tree, as these methods do not scale well.

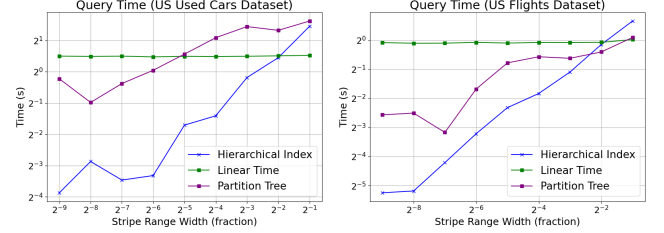


Figure 6: Comparison of the query time of Hierarchical Sampling on SRR problem vs the stripe range width.

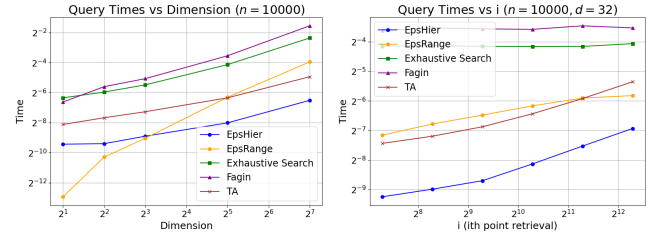


Figure 7: Comparison of the query time of EpsRange and EpsHier on κ -RAR problem vs dimension d and the rank i . In this setting, $\epsilon = \frac{1}{16}$.

observe that both KD-Tree and R-Tree methods fail to scale effectively to high-dimensional settings, and thus we exclude them from experiments with $d = 128$. The proposed Hierarchical Sampling method achieves **up to a 16 $\times$ speedup** over Exhaustive search, particularly in *narrow stripe* queries. Such narrow stripes frequently arise when solving the RAR problem, indicating that this method is also helpful in this context. While the Partition Tree also shows a fast query time in low-dimensional spaces, its performance degrades with increasing dimensionality.

Effect of Dataset Size. Figure 5 illustrates the impact of increasing the dataset size n on the query time of the algorithms, for both low- and high-dimensional settings. As observed, the Hierarchical Sampling method consistently outperforms the baselines.

Real Datasets. Figure 6 shows the comparison on US USED CARDS dataset. As shown, Hierarchical Sampling significantly outperforms both Exhaustive search and the Partition Tree method. In this experiment, multiple random scoring functions are sampled uniformly from the unit hypersphere. (See Appendix M.2 for other real-datasets).

Accuracy of the Output. In all the settings, the output is guaranteed to have a recall of 100%, reporting all the points in the range.

¹³The number in the dataset name represents the dimensionality of the vectors.

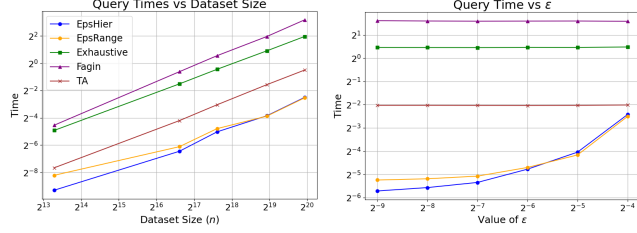


Figure 8: Comparison of the query time of EpsRange and EpsHier on κ -RAR problem vs dataset size and ϵ . In the right plot, $n = 10000$ and $\kappa = \epsilon n$.

7.2 Random-Access Ranked Retrieval

We include the Threshold Algorithm (TA) and Fagin’s algorithm [44] as baseline methods for retrieving top-K elements.¹⁴ Additionally, we consider a simple search baseline, referred to as Exhaustive, which computes all the scores and extracts the i -th ranked element. Our proposed algorithms (EpsRange and EpsHier) are used for the κ -RAR problem while the KthLevel algorithm solves the exact 1-RAR problem in 2D (see Appendix M.3).

7.2.1 Synthetic Dataset.

Effect of Dimension and value of i . Figure 7 (left) presents the query time of algorithms as a function of dimension d . As d increases, both Fagin and TA algorithms exhibit poor scalability for random-access queries. Among the proposed methods, EpsHier consistently outperforms EpsRange, particularly when the $d > 8$. The performance degradation of EpsRange in higher dimensions is because of its reliance on Partition Tree index [72], which suffers from the curse of dimensionality. Figure 7 (right) illustrates the impact of varying the value of i in the RAR query on the runtime. The TA algorithm shows increasing query time as i grows, since it must retrieve all the top- i items. In contrast, the runtime of EpsRange remains relatively stable, as it depends primarily on the position of the stripe rather than the value of i . The EpsHier method also shows a moderate increase in runtime for larger i , which can be because of the relative location of the stripes with respect to the dataset. The marginal stripes (smaller values of i) intersect with fewer balls in the hierarchical structure proposed in Section 6.3, and the pruning is more effective in this case.

Effect of Dataset Size and ϵ . Figure 8 (left) shows the effect of dataset size on the query time of the algorithms. We observe significant speedups achieved by both EpsRange and EpsHier compared to the baseline methods. Figure 8 (right) illustrates the impact of varying the parameter ϵ . As expected, this parameter influences the size of the output set, κ , in the κ -RAR problem. Increasing the value of ϵ results in a larger output set, whereas smaller values of ϵ require searching over a larger ϵ -sample during the initial phase of the algorithm. Both extremes lead to increased query times. The most efficient performance is typically achieved for a moderate value of ϵ , which may serve as a default when not given by user.

Real Datasets. Figure 9 presents the results of running the algorithms on the US Flights dataset, evaluating across different values of ϵ (left) and different values of i (right). We observe significant speedups achieved by EpsHier, followed by EpsRange. Appendix M.3 shows the results on other datasets.

¹⁴Please refer to the related work (Appendix B).

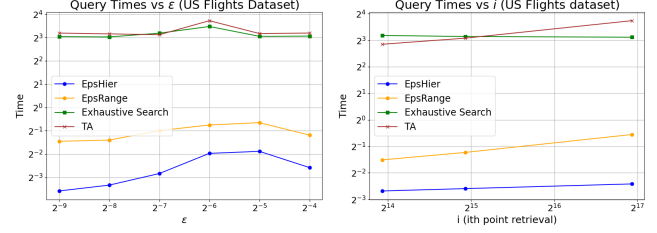


Figure 9: Comparing the query time of EpsHier and EpsRange on κ -RAR problem vs the value of ϵ and rank i on US Flights dataset.

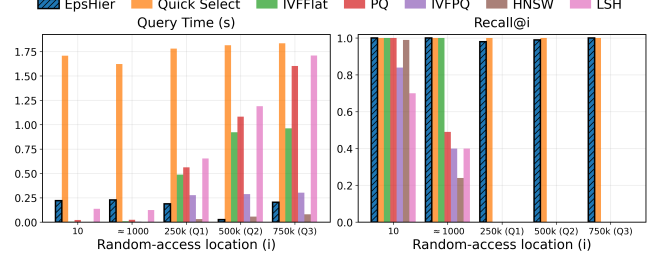


Figure 10: Comparing query time and recall@ i on random-access similarity search task. The dataset is SIFT-128-EUCLIDEAN with 1M vectors of dimensionality 128 with ℓ_2 distance.

Accuracy of the Output. In all the experiments, the returned set is guaranteed to have the $p^{(i)}$, so the recall is always 100%.

7.3 Random-Access Similarity Search

For the similarity search setting, we first apply the reduction described in Section 3.4. For datasets under the ℓ_2 metric, we lift the data points during preprocessing and apply the same lifting to the query at query time. For cosine similarity, we treat the query as a linear scoring function corresponding to the standard dot product. The objective is to retrieve the i -th nearest neighbor of a query point, where i is not necessarily $O(1)$.

We consider six baselines for comparison. Our first baseline is (i) the classical *Quickselect* algorithm for order statistics computation.¹⁵ We also evaluate several popular ANN indexes implemented in the widely-used FAISS library [40]: (ii) HNSW [70], (iii) PQ [58], (iv) IVFPQ [58], (v) IVFFlat [58], and (vi) LSH [7]. The details of these algorithms are discussed in the related works (Appendix B).

In Appendix M.4, we describe how these ANN indexes are adapted to support the random-access version of the problem. We note that, as demonstrated by our experimental results, these methods are generally not well suited for random-access retrieval when i (or equivalently k) is large; they are included primarily for completeness and comparison.

Figure 10 shows the recall and query time of our algorithm, EpsHier, compared with the baseline methods on the SIFT-128-EUCLIDEAN dataset. As the target rank i increases to larger values (e.g., the first, second, and third quartiles), all ANN-based methods degrade substantially, exhibiting both increased query time and near-zero recall. In contrast, EpsHier consistently achieves close to 100% recall while maintaining relatively low query time, even when random-accessing elements in the middle of the ranked order. This behavior is particularly evident for large ranks $i = O(n)$. Table 2 in Appendix M.4 shows a complete comparison on a wide range of i values and other datasets.

¹⁵<https://en.wikipedia.org/wiki/Quickselect>

8 Conclusion

We studied the problem of Random-Access Ranked Retrieval (RAR) and Random-Access Similarity Search (RAS). We showed that RAS can be reduced to RAR. For solving the general RAR problem, we first proposed an algorithm based on geometric arrangements with logarithmic query time but an exponential space complexity. Next, we proposed our space-efficient algorithms based on ε -samples. By providing a hierarchical sampling-based data structure for efficient stripe-range retrieval, we developed a practical algorithm for the relaxed κ -RAR problem that scales to hundreds of dimensions.

References

- [1] [n.d.]. CSMetrics. <https://csmetrics.net/>. [Online; accessed Feb. 2026].
- [2] Pankaj K Agarwal. 2017. Range searching. In *Handbook of discrete and computational geometry*. Chapman and Hall/CRC, 1057–1092.
- [3] Pankaj K Agarwal et al. 1998. Constructing levels in arrangements and higher order Voronoi diagrams. *SIAM journal on computing* 27, 3 (1998), 654–667.
- [4] Pankaj K Agarwal, Ravid Cohen, Dan Halperin, and Wolfgang Mulzer. 2019. Dynamic maintenance of the lower envelope of pseudo-lines. *arXiv preprint arXiv:1902.09565* (2019).
- [5] Alok Aggarwal, Mark Hansen, and Thomas Leighton. 1990. Solving query-retrieval problems by compacting Voronoi diagrams. In *Proceedings of the twenty-second annual ACM symposium on Theory of computing*. 331–340.
- [6] Yongsu Ahn and Yu-Ru Lin. 2019. Fairsight: Visual analytics for fairness in decision making. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 1086–1095.
- [7] Alexandr Andoni and Piotr Indyk. 2008. Near-optimal hashing algorithms for approximate nearest neighbor in high dimensions. *Communications Of ACM* 51, 1 (2008), 117–122.
- [8] Anastasios N Angelopoulos and Stephen Bates. 2021. A gentle introduction to conformal prediction and distribution-free uncertainty quantification. *arXiv preprint arXiv:2107.07511* (2021).
- [9] Abolfazl Asudeh, HV Jagadish, Gerome Miklau, and Julia Stoyanovich. 2018. On obtaining stable rankings. *Proceedings of the VLDB Endowment* 12, 3 (2018), 237–250.
- [10] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. 2018. ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms. *arXiv:1807.05614* [cs.LG]. <https://arxiv.org/abs/1807.05614>
- [11] Alex Auvolet et al. 2015. Clustering is efficient for approximate maximum inner product search. *arXiv preprint arXiv:1507.05910* (2015).
- [12] Yoram Bachrach et al. 2014. Speeding up the xbox recommender system using a euclidean transformation for inner-product spaces. In *Proceedings of the 8th ACM Conference on Recommender systems*. 257–264.
- [13] R Ricardo Baeza-Yates et al. 1999. *Modern information retrieval*. ACM Press.
- [14] Guillaume Bagan, Arnaud Durand, Etienne Grandjean, and Frédéric Olive. 2008. Computing the j th solution of a first-order query. *RAIRO-Theoretical Informatics and Applications* 42, 1 (2008), 147–164.
- [15] Ahmet Zahid Balcioglu and Oğuz Güreker. 2022. On a Notion of Outliers Based on Ratios of Order Statistics. *arXiv preprint arXiv:2207.13068* (2022).
- [16] Jon Louis Bentley. 1975. Multidimensional binary search trees used for associative searching. *Commun. ACM* 18, 9 (1975), 509–517.
- [17] Nikos Bikakis, Panos K Chrysanthis, Guoliang Li, George Papastefanatos, and Lingyun Yu. 2025. Visual Analytics Challenges and Trends in the Age of AI: The BigVis Community Perspective. *ACM SIGMOD Record* 54, 2 (2025), 66–69.
- [18] Manuel Blum et al. 1973. Time bounds for selection. *J. Comput. Syst. Sci.* 7, 4 (1973), 448–461.
- [19] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems* 30, 1-7 (1998), 107–117.
- [20] Karl Bringmann, Nofar Carmeli, and Stefan Mengel. 2025. Tight fine-grained bounds for direct access on join queries. *ACM Transactions on Database Systems* 50, 1 (2025), 1–44.
- [21] Andrei Broder. 2002. A taxonomy of web search. In *ACM Sigir forum*, Vol. 36. ACM New York, NY, USA, 3–10.
- [22] N. Bruno, L. Gravano, and A. Marian. 2002. Evaluating top-k queries over Web-accessible databases. In *IEEE ICDE*. 369–380.
- [23] Nofar Carmeli, Nikolaos Tziavelis, Wolfgang Gatterbauer, Benny Kimelfeld, and Mirek Riedewald. 2023. Tractable orders for direct access to ranked answers of conjunctive queries. *ACM Transactions on Database Systems* 48, 1 (2023), 1–45.
- [24] Timothy M Chan. 2020. Dynamic geometric data structures via shallow cuttings. *Discrete & Computational Geometry* 64, 4 (2020), 1235–1252.
- [25] Yuan-Chi Chang, Lawrence Bergman, Vittorio Castelli, Chung-Sheng Li, Ming-Ling Lo, and John R. Smith. 2000. The onion technique: indexing for linear optimization queries. *ACM SIGMOD* 29, 2 (2000), 391–402.
- [26] Bernard Chazelle. 1989. Lower bounds on the complexity of polytope range searching. *Journal of the American Mathematical Society* 2, 4 (1989), 637–666.
- [27] Bernard Chazelle. 2000. *The Discrepancy Method: Randomness and Complexity*. Cambridge University Press.
- [28] Bernard Chazelle and Leonidas J Guibas. 1986. Fractional cascading: I. A data structuring technique. *Algorithmica* 1, 1 (1986), 133–162.
- [29] Bernard Chazelle, Leo J Guibas, and Der-Tsai Lee. 1985. The power of geometric duality. *BIT Numerical Mathematics* 25, 1 (1985), 76–90.
- [30] Fanchao Chen, Dixin Tang, Haotian Li, and Aditya G Parameswaran. 2023. Visualizing Spreadsheet Formula Graphs Compactly. *Proceedings of the VLDB Endowment* 16, 12 (2023), 4030–4033.
- [31] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. 2022. *Introduction to algorithms*. MIT press.
- [32] Graham Cormode, Zohar Karnin, Edo Liberty, Justin Thaler, and Pavel Vesely. 2021. Relative error streaming quantiles. In *Proceedings of the 40th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 96–108.
- [33] W Bruce Croft, Donald Metzler, Trevor Strohman, et al. 2010. *Search engines: Information retrieval in practice*. Vol. 520. Addison-Wesley Reading.
- [34] Gautam Das. 2009. Top-k Algorithms and Applications. In *DASFAA*. 789–792.
- [35] Gautam Das, Dimitrios Gunopulos, Nick Koudas, and Dimitris Tsirogiannis. 2006. Answering top-k queries using views. In *VLDB*. 451–462.
- [36] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *SCG*. 253–262.
- [37] Mohsen Dehghankar and Abolfazl Asudeh. 2025. HENN: A Hierarchical Epsilon Net Navigation Graph for Approximate Nearest Neighbor Search. *CoRR, abs/2505.17368* (2025).
- [38] Tamal K Dey. 1998. Improved bounds for planar k-sets and related problems. *Discrete & Computational Geometry* 19, 3 (1998), 373–382.
- [39] David Dobkin, John Hershberger, David Kirkpatrick, and Subhash Suri. 1990. Implicitly searching convolutions and computing depth of collision. In *International Symposium on Algorithms*. Springer, 165–180.
- [40] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. (2024). *arXiv:2401.08281* [cs.LG]
- [41] Herbert Edelsbrunner. 1987. *Algorithms in combinatorial geometry*. Vol. 10. Springer Science & Business Media.
- [42] Idan Eldar, Nofar Carmeli, and Benny Kimelfeld. 2023. Direct access for answers to conjunctive queries with aggregation. *arXiv preprint arXiv:2303.05327* (2023).
- [43] Hazel Everett et al. 1993. An optimal algorithm for the ($\leq k$)-levels, with applications to separation and transversal problems. In *Proceedings of the ninth annual symposium on Computational geometry*. 38–46.
- [44] Ronald Fagin, Amnon Lotem, and Moni Naor. 2001. Optimal aggregation algorithms for middleware. In *ACM PODS*. 102–113.
- [45] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast approximate nearest neighbor search with the navigating spreading-out graph. *PVLDB* 12, 5 (2019), 461–474.
- [46] Dhruvajyoti Ghosh, Peeyush Gupta, Sharad Mehrotra, Roberto Yus, and Yasser Altowim. 2022. JENNER: just-in-time enrichment in query processing. *Proceedings of the VLDB Endowment* 15, 11 (2022).
- [47] Phillip B Gibbons, Yossi Matias, and Viswanath Poosala. 2002. Fast incremental maintenance of approximate histograms. *ACM Transactions on Database Systems (TODS)* 27, 3 (2002), 261–298.
- [48] Michael Greenwald and Sanjeev Khanna. 2001. Space-efficient online computation of quantile summaries. *ACM SIGMOD Record* 30, 2 (2001), 58–66.
- [49] Antonin Guttman. 1984. R-trees: A dynamic index structure for spatial searching. In *ACM SIGMOD*. 47–57.
- [50] Sarel Har-Peled. 2011. *Geometric approximation algorithms*. Number 173. American Mathematical Soc.
- [51] David Haussler and Emo Welzl. 1986. Epsilon-nets and simplex range queries. In *Proceedings of the second annual symposium on Computational geometry*. 61–71.
- [52] Vagelis Hristidis et al. 2004. Algorithms and applications for answering ranked queries using ranked views. *The VLDB Journal* 13, 1 (2004), 49–70.
- [53] Ihab F Ilyas et al. 2008. A survey of top-k query processing techniques in relational database systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 1–58.
- [54] Ihab F. Ilyas et al. 2008. A survey of top-k query processing techniques in relational database systems. *Comput. Surveys* 40, 4, Article 11 (2008).
- [55] Piotr Indyk and Rajeev Motwani. 1998. Approximate nearest neighbors: towards removing the curse of dimensionality. In *ACM STOC*. 604–613.
- [56] Omid Jafari, Preeti Maurya, Parth Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942* (2021).
- [57] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *NeurIPS*, Vol. 32.
- [58] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE transactions on pattern analysis and machine intelligence* 33, 1 (2010), 117–128.

- [59] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2019), 535–547.
- [60] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.
- [61] Zohar Karnin, Kevin Lang, and Edo Liberty. 2016. Optimal quantile approximation in streams. In *2016 IEEE 57th annual symposium on foundations of computer science (focs)*. IEEE, 71–78.
- [62] Thomas Kluyver et al. 2016. Jupyter Notebooks—a publishing format for reproducible computational workflows. In *Positioning and power in academic publishing: Players, agents and agendas*. IOS press, 87–90.
- [63] Stefano Leone. 2022. FIFA 23 Player Dataset. <https://www.kaggle.com/datasets/stefanoleone992/fifa-23-complete-player-dataset>. Accessed: 2025-07-31.
- [64] Justin J Levandoski, Mohamed F Mokbel, and Mohamed E Khalefa. 2010. Preference query evaluation over expensive attributes. In *Proceedings of the 19th ACM international conference on Information and knowledge management*. 319–328.
- [65] Huajing Li, Yuan Tian, Wang-Chien Lee, C Lee Giles, and Meng-Chang Chen. 2010. Personalized feed recommendation service for social networks. In *2010 IEEE Second International Conference on Social Computing*. IEEE, 96–103.
- [66] Zhonghang Liu, Panzhong Lu, Guoyang Xie, Zhichao Lu, and Wen-Yan Lin. 2024. Rethinking unsupervised outlier detection via multiple thresholding. In *European Conference on Computer Vision*. Springer, 258–275.
- [67] Qin Lv, William Josephson, Zhe Wang, Moses Charikar, and Kai Li. 2007. Multi-probe LSH: efficient indexing for high-dimensional similarity search. In *VLDB*. 950–961.
- [68] Zeyao Ma et al. 2024. Spreadsheetbench: Towards challenging real world spreadsheet manipulation. *Advances in Neural Information Processing Systems* 37 (2024), 94871–94908.
- [69] Yury Malkov, Alexander Ponomarenko, Andrey Logvinov, and Vladimir Krylov. 2014. Approximate nearest neighbor algorithm based on navigable small world graphs. *Information Systems* 45 (2014), 61–68.
- [70] Yu A Malkov and Dmitry A Yashunin. 2018. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence* 42, 4 (2018), 824–836.
- [71] Jiří Matoušek. 1991. Efficient partition trees. In *Proceedings of the seventh annual symposium on Computational geometry*. 1–9.
- [72] Jiri Matousek. 1992. Reporting points in halfspaces. *Computational Geometry* 2, 3 (1992), 169–186.
- [73] Stefan Meiser. 1993. Point location in arrangements of hyperplanes. *Information and Computation* 106, 2 (1993), 286–303.
- [74] Chuan Meng, Negar Arabzadeh, Arian Askari, Mohammad Aliannejadi, and Maarten De Rijke. 2024. Ranked list truncation for large language model-based re-ranking. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval*. 141–151.
- [75] Ananay Mital. 2020. US Used Cars Dataset. <https://www.kaggle.com/datasets/ananyamital/us-used-cars-dataset>. Accessed: 2025-07-31.
- [76] Douglas C Montgomery, Elizabeth A Peck, and G Geoffrey Vining. 2021. *Introduction to linear regression analysis*. John Wiley & Sons.
- [77] Stephen M Omohundro. 1989. Five balltree construction algorithms. (1989).
- [78] Serhii Orlivskyi, Bohdan Deomin, and Olga Averianova. 2021. Pagination And Its Efficient Methods For RESTful Web Services. In *2021 IEEE 3rd Ukraine Conference on Electrical and Computer Engineering (UKRCON)*. IEEE, 567–571.
- [79] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615.
- [80] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Empirical Methods in Natural Language Processing (EMNLP)*. 1532–1543. <http://www.aclweb.org/anthology/D14-1162>
- [81] Franco P Preparata and Michael I Shamos. 2012. *Computational geometry: an introduction*. Springer Science & Business Media.
- [82] Sajjadur Rahman et al. 2020. Benchmarking spreadsheet systems. In *Proceedings of the 2020 ACM SIGMOD*. 1589–1599.
- [83] Carl M Rebman Jr et al. 2023. An Industry Survey of Analytics Spreadsheet Tools Adoption: Microsoft Excel vs Google Sheets. *Information Systems Education Journal* 21, 5 (2023), 29–42.
- [84] Francesco Ricci et al. 2021. Recommender systems: Techniques, applications, and challenges. *Recommender systems handbook* (2021), 1–35.
- [85] Glenn Shafer and Vladimir Vovk. 2008. A tutorial on conformal prediction. *Journal of Machine Learning Research* 9, 3 (2008).
- [86] Shreya Shankar, Stephen Macke, Sarah Chasins, Andrew Head, and Aditya Parameswaran. 2022. Bolt-on, compact, and rapid program slicing for notebooks. *Proceedings of the VLDB Endowment* 15, 13 (2022), 4038–4047.
- [87] Micha Sharir, Shakhar Smorodinsky, and Gábor Tardos. 2000. An improved bound for k-sets in three dimensions. In *Proceedings of the sixteenth annual symposium on Computational geometry*. 43–49.
- [88] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for sublinear time maximum inner product search (MIPS). *Advances in neural information processing systems* 27 (2014).
- [89] Nisheeth Shrivastava, Chiranjeev Buragohain, Divyakant Agrawal, and Subhash Suri. 2004. Medians and beyond: new aggregation techniques for sensor networks. In *Proceedings of the 2nd international conference on Embedded networked sensor systems*. 239–249.
- [90] Parsharam Reddy Sudda. 2024. Optimizing PHP API calls with pagination and caching. (2024).
- [91] Martin Theobald, Gerhard Weikum, and Ralf Schenkel. 2004. Top-k query evaluation with probabilistic guarantees. In *VLDB*. 648–659.
- [92] Godfried T. Toussaint. 1980. The relative neighbourhood graph of a finite planar set. *Pattern Recognition* 12, 4 (1980), 261–268.
- [93] Nikolaos Tziavelis, Wolfgang Gatterbauer, and Mirek Riedewald. 2024. Ranked enumeration for database queries. *ACM SIGMOD Record* 53, 3 (2024), 6–19.
- [94] Bureau of Transportation Statistics U.S. Department of Transportation. n.d.. Flight Delays and Cancellations. <https://www.kaggle.com/datasets/usdot/flight-delays?select=flights.csv>. Accessed: 2025-10-04.
- [95] Vladimir N Vapnik and A Ya Chervonenkis. 2015. On the uniform convergence of relative frequencies of events to their probabilities. In *Measures of complexity: festschrift for alexey chervonenkis*. Springer, 11–30.
- [96] Jeffrey S Vitter. 1985. Random sampling with a reservoir. *ACM Transactions on Mathematical Software (TOMS)* 11, 1 (1985), 37–57.
- [97] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. *arXiv preprint arXiv:2101.12631* (2021).
- [98] Emo Welzl. 2005. Smallest enclosing disks (balls and ellipsoids). In *New Results and New Trends in Computer Science: Graz, Austria, June 20–21, 1991 Proceedings*. Springer, 359–370.
- [99] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. *Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:cs.LG/1708.07747 [cs.LG]
- [100] Jinjin Zhao et al. 2023. Data Makes Better Data Scientists. In *Proceedings of the Workshop on Human-In-the-Loop Data Analytics*. 1–3.
- [101] Zeyang Zhu, Jonathan Chamberlain, Kenny Wu, David Starobinski, and Zaoxing Liu. 2025. Approximation-First Timeseries Monitoring Query At Scale. *arXiv preprint arXiv:2505.10560* (2025).

Appendix

Table of Content

1. More on the Application Examples	A
2. Related Work	B
3. Extension to Non-linear Scoring Functions	C
4. Missing Details for Reducing RAS to RAR	D
5. Our Algorithm Based on k -th Level of Arrangement	E
6. Analysis of the Eps2D Algorithm	F
7. Missing Proofs	G
8. Dynamic Setting	H
9. Analysis of the EPSRANGE Algorithm	I
10. Analysis of Hierarchical Sampling	J
11. Pseudo-codes	K
12. Lower Bounds and Optimality of Our Algorithms	L
13. More on the Experiments	M

A More on the Application Examples

Examples 1 and 2 highlight two motivations for random-access ranked retrieval for e-commerce and web applications. Such examples can be found across a wider range of domains. In this section, we highlight additional application examples of RAR and its variations across a diverse set of domains.

Extending random-access to ranked retrieval. Random access to arbitrary positions in an array or a sorted list is a fundamental operation underlying many of the most efficient algorithms in computer science. Classic divide-and-conquer techniques, such as **binary search** and **quick select**, critically rely on the ability to randomly access elements at arbitrary positions in order to efficiently prune the search space.

However, in ranked retrieval systems, this basic capability is largely missing. Although items are conceptually ordered by a scoring or ranking function, accessing the element at a given rank typically requires enumerating all higher-ranked items, effectively reducing random access to sequential scan. This limitation prevents the direct application of well-established algorithmic techniques in settings where ordering is induced by a query-dependent ranking rather than a static key.

The random-access ranked retrieval (RAR) problem addresses this gap by providing efficient access to arbitrary rank positions under a ranking function without materializing the entire result prefix. RAR extends the power of random-access-based algorithms to settings where the values are scores, determined by a ranking function or by similarity to a specific point. This capability opens the door to adapting a broad class of classical algorithms, previously confined to static (sorted) lists, to modern data retrieval workloads where rankings are query-specific (dynamic) and high-dimensional.

Analytical Insights for Business Decision Support. In many business and decision-support applications, stakeholders are interested in advanced analytics (e.g., income distribution, demographic distribution, shopping behavior) for not only in the top-ranked customers, or the ones in their close vicinity, but in understanding how key analytics look across different rank positions or percentiles of a scoring function [32, 48, 89, 101]. Examples include analyzing income or demographic distributions across distance bands from

a retail location, studying customer engagement at different relevance thresholds, or evaluating how small changes in targeting criteria affect the selected population.

Consider a targeted advertising scenario in which a business defines its audience using a distance-based or machine-generated ranking function – for instance, customers ranked by proximity to a store or by predicted likelihood of conversion. While traditional systems efficiently retrieve the top- k customers, business decisions often hinge on understanding boundary cases: customers near a cut-off who are just included or excluded under a given targeting policy. These boundary regions are especially important for evaluating trade-offs, fairness, and return on investment.

Supporting such analyses requires efficient access to records at arbitrary quantiles or rank positions (e.g., “the 90th-percentile customer according to this scoring function”), as well as the ability to explore narrow rank intervals around these positions. Naïve approaches based on sampling or top- k retrieval are ineffective, since points at the same rank or distance may be widely dispersed in the feature space and cannot be approximated by local neighborhoods.

Random-access ranked retrieval enables this form of fine-grained analytical exploration by allowing analysts to directly probe specific rank positions and score bands without exhaustively scanning the dataset. As a result, it provides a powerful primitive for interactive business analytics, sensitivity analysis, and decision support in large-scale, high-dimensional settings.

Efficient Database Queries over Derived Attributes. Modern database systems routinely support queries over *derived attributes*, i.e., attributes that are not explicitly stored in the database but are computed on-the-fly as part of a query [46, 64]. Such derived attributes are commonly expressed as functions over existing columns and are treated as first-class citizens in query predicates, projections, and orderings. For example, consider the following SQL query:

```
SELECT Name, YEAR(Today_Date) - YEAR(Birth_Date) AS Age
FROM Employees
WHERE Age BETWEEN 25 AND 30;
```

Here, Age is not a stored attribute but is defined by the query itself as a function of Birth_Date. Conceptually, this corresponds to introducing a new attribute whose value is computed per tuple and then used for filtering or ranking.

From a ranked-retrieval perspective, derived attributes naturally induce query-dependent scoring functions that combine existing attributes through arithmetic or functional transformations. Supporting efficient access to tuples based on such derived attributes poses a fundamental challenge. Since derived attributes do not exist at indexing time, traditional indexing techniques cannot be applied directly, and a naïve solution requires computing the derived value for all tuples. This is particularly expensive when users wish to explore arbitrary rank positions or specific rank ranges. Also in spreadsheet applications [68, 83], data scientists frequently create new attributes as weighted sums of existing ones. Subsequently, they may “jump” to tuples at specific rank positions without explicitly enumerating preceding entries.

By treating derived attributes as scoring functions defined at query time, RAR enables efficient access to arbitrary rank positions without fully materializing or sorting the entire result set. This makes RAR a natural foundation for supporting interactive and

scalable database queries over derived attributes, extending classical database functionality with modern rank-aware access patterns.

Individually Fair Decision Boundaries. In many decision-making scenarios, the objective is not merely to retrieve items at a fixed rank, but to identify the cut-off points in a ranked list as the decision boundaries [6, 15, 66, 74]. To minimize individual unfairness (i.e., to produce different outputs for similar inputs), the cut-off points should indicate meaningful transitions between the classes. In other words, the gap at the boundary should be wide. A tangible example arises in educational assessment, where instructors must determine grade boundaries (e.g., between an A and a B) based on student scores. Rather than enforcing a predefined percentile or absolute threshold, instructors often examine the score distribution around a target rank (e.g., the 90th percentile) to identify natural breaks that separate performance tiers. Similarly, drawing fair decision boundaries in score-based data-driven systems (e.g., for loan applications) requires exploring a band (range) of valid cut-off points to maximize the decision-boundary gap [6].

Random-access ranked retrieval enables principled gap discovery by supporting direct, localized inspection of ranked items around candidate cut-off points. This capability allows decision makers to efficiently retrieve entities within the decision boundary candidates' range and to set the cut-off point responsibly.

B Related Work

Top- k Retrieval. Top- k query processing has been extensively studied across various settings. The foundational Threshold Algorithm (TA) [44] combines sorted and random access to efficiently identify top- k results and has inspired numerous extensions, including cost-based optimizations and early stopping strategies [22]. Probabilistic top- k methods [91] adapt this problem to uncertain data, ranking results based on expected scores or statistical confidence. Index-based approaches such as the onion layer [25] and view-based techniques [35, 52] have been proposed to accelerate queries through structural or materialized reuse. For comprehensive surveys, see [34, 54]. *In top- k retrieval, k is a small constant*, while in our setting, we aim to find the tuple at rank position i , where i is $O(n)$. As a result, the algorithms proposed for top- k are inefficient for our setting, as shown in the experiments.

Direct Access Queries in Databases. Recent work in database theory has explored the notion of *direct access* to query answers, aiming to support efficient access to the i -th ranked tuple without materializing the full result [14, 20, 23, 42, 93]. In these works, the ordering is generally assumed to be fixed, lexicographically, on some attributes. Some work extends this to conjunctive queries with aggregation, designing algorithms that enable access to top-ranked answers without fully evaluating the query [42]. In our paper, we discuss any ranking function defined as the weights on the features. Specifically, rather than assuming a predefined ordering, *we consider a ranking function as part of the user query*. As a result, the proposed algorithms for direct-access queries cannot be adapted to our setting.

Range Searching Indexes. Range searching, specifically for simplex ranges, is related to our work in solving the intermediate SRR problem. Some approaches achieve logarithmic query time at

the cost of exponential space [5, 39], while others aim for linear space and sublinear query time [29, 51, 71, 72]. Specialized variants have also been studied, including half-space range searching [29] and axis-aligned queries [28]. For a comprehensive survey, see [2]. To address the SRR problem in practice, we develop a hierarchical sampling approach inspired by data structures commonly used for similarity search, like HNSW [70]. As an intermediate problem for solving RAR, *our solutions are tailored for narrow ranges*. While our methods do not offer formal theoretical guarantees, they performed efficiently in practical settings. Another indexing technique related to our algorithm is the *Ball Tree* structure [77]. The key difference between Ball Trees and our approach lies in the construction and structure. In our method, each layer is generated by randomly sampling from the previous one, and each layer serves as a set of centroids for the preceding layer. In contrast, Ball Trees are typically binary trees built in a top-down manner.

Other related topic is *Max Inner Product Search (MIPS)* [12, 88] which focuses on retrieving the item with the highest inner product with a query vector. Techniques, such as asymmetric LSH [88] and tree- or graph-based indices [11, 12], have been proposed to accelerate MIPS. In parallel, *Quantile Sketches* [48, 61] provide compact data summaries that approximate rank-based statistics, enabling efficient quantile estimation in streaming or large-scale settings. Unlike these methods, our work on *Random-Access Retrieval* aims to directly access the i -th element according to an arbitrary scoring function.

Nearest Neighbor Search. A closely related problem to random-access similarity search (Problem 3) is *Approximate Nearest Neighbor (ANN)* search, which aims to retrieve the top- k most similar points to a query q . ANN has been extensively studied in the context of vector databases [10, 55], and existing methods broadly fall into three categories: (a) graph-based indexing and traversal methods, (b) quantization-based approaches that compress the search space, and (c) hashing techniques inspired by locality-sensitive hashing.

Graph-based methods construct a proximity graph over the dataset and perform greedy graph traversal at query time [81, 92]. Prominent examples include Navigable Small-World (NSW) graphs and their hierarchical variant HNSW [69, 70], as well as Navigating Spread-out Graphs (NSG) [45] and DiskANN [57].

Quantization-based techniques aim to accelerate kNN search by compressing high-dimensional vectors. Widely used approaches include Product Quantization (PQ) [60] and Inverted File (IVF) indexing [40, 59]. PQ and its variants decompose vectors into multiple low-dimensional subspaces and quantize each independently, enabling fast approximate distance computation via lookup tables. For large-scale datasets, PQ is commonly combined with coarse partitioning schemes such as IVF, which clusters the dataset into centroid-based cells approximating Voronoi regions.

The third category consists of hash-based methods such as locality-sensitive hashing, which provide theoretical guarantees [7, 36, 67] but are known to degrade in high-dimensional real-world settings [56].

While these methods are designed to solve similarity search under distance metrics such as ℓ_2 or cosine similarity, they primarily target regimes where k is a small constant. As demonstrated in our experiments, their performance degrades substantially when

applied to random-access retrieval at an arbitrary rank i . In contrast, our algorithm is explicitly designed to address this setting.

C Extension to Non-linear Scoring Functions

In Section 3, we defined ranked retrieval with respect to *linear scoring functions*, where the score of a point $p \in \mathbb{R}^d$ is computed as a weighted sum of its attributes, i.e.,

$$\text{score}_f(p) = f^\top p = \sum_{j=1}^d f[j]p[j],$$

for a weight vector $f \in \mathbb{R}^d$.

While our formal development focuses on linear scoring functions, the scope of our framework naturally extends to a broad class of non-linear scoring functions that can be expressed as a *linear combination of derived terms*. Such representations are standard in machine learning and data analysis, including linear regression models with feature transformations [76] and monotonic transformations commonly used in ranking systems [9].

The key idea is to introduce non-linear terms as additional *derived attributes*, thereby lifting the original feature space into a higher-dimensional space in which the scoring function becomes linear. All results developed for linear scoring functions are then applied directly in the expanded space.

Illustrative Example. Consider the following non-linear scoring function over two attributes $\{x, y\}$:

$$\text{score}(p) = 2x^2 + y^2 - 3x/y + 5xy - x + 2y.$$

This function can be equivalently expressed as a linear scoring function over the expanded attribute set

$$\{x_1 = x^2, x_2 = y^2, x_3 = x/y, x_4 = xy, x_5 = x, x_6 = y\}.$$

Then, using the new attributes, the function can be written as

$$\text{score}(p) = 2x_1 + x_2 - 3x_3 + 5x_4 - x_5 + 2x_6.$$

Real-World Example. As a concrete real-world instance, consider the ranking function used by CSMetrics [1] to rank computer science research institutions. The ranking combines two attributes for each institution: the number of *measured* citations (M) and an estimate of *predicted* future citations (P). Given a parameter $\alpha \in [0, 1]$, the scoring function is defined as

$$\text{score}_\alpha(p) = (M)^\alpha (P)^{1-\alpha}.$$

Although this function is multiplicative and non-linear in the original attributes, it can be transformed into a linear form by introducing the derived attributes [9]

$$x_1 = \log(M), \quad x_2 = \log(P),$$

Using these attributes, the scoring function can be rewritten as

$$\text{score}_\alpha^\ell(p) = \alpha x_1 + (1 - \alpha)x_2.$$

Since the logarithm is a monotonic transformation, the original function $\text{score}_\alpha(\cdot)$ and its linearized form $\text{score}_\alpha^\ell(\cdot)$ induce identical rankings. Consequently, all algorithms and guarantees developed in this paper apply unchanged to such non-linear scoring functions after feature transformation.

D Missing Details for Reducing RAS to RAR

D.1 Euclidean Distance: Proof of Lemma 1

PROOF. Consider the Euclidean distance function

$$\mathbf{d}(p, q) = \|p - q\|_2 = \sqrt{\sum_{j=1}^d (p[j] - q[j])^2}.$$

We first note that, instead of using $\mathbf{d}(p, q)$, one could use $\mathbf{d}(p, q)^2$ as the function without loss of generality.

The expansion of the term $\mathbf{d}(p, q)^2$ provides the basis of a transformation that connects RAS and CSS problems. This expansion is given by,

$$\|p - q\|_2^2 = \|p\|_2^2 - 2q^\top p + \|q\|_2^2 \quad (5)$$

The term $q^\top \cdot p$ is akin to computing the score based on the dot product (i.e., a linear function), where q is the weight vector.

The d dimensional RAS problem can be reduced to the $d + 1$ dimensional RAR problem using a transformation. The transformation relies on the notion of Paraboloid lifting [50] given by $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d+1}$. The function ϕ transforms a d dimensional point to a $d + 1$ dimensional point in a new transformed space. Following literature, the paraboloid lifting is defined as below,

$$\phi(p) = (p[1], p[2], \dots, p[d], \|p\|_2^2) \quad (6)$$

As a first step, during the pre-processing stage, the paraboloid lifting function ϕ is applied to every point in the dataset $\mathcal{D}$ to obtain $d + 1$ dimensional dataset. When an RAS query q arrives, we transform the query $q \in \mathbb{R}^d$ to a $d + 1$ dimensional RAR scoring function $f = (-2q[1], -2q[2], \dots, -2q[d], 1)$. Applying the scoring function f to any point p in the dataset, we obtain the term

$$\text{score}_f(p) = f^\top p = \|p\|_2^2 - 2q^\top p = \|p - q\|_2^2 - \|q\|_2^2$$

Note that the term $\|q\|_2^2$ is still needed to obtain the exact same value as the $\mathbf{d}(p, q)^2$ term in Equation 5. But, as the term $\|q\|_2^2$ is independent of p , and is missing for all $p_i \in \mathcal{D}$ as part of the computation of f , there is no change in ranking. That is, while the scores for all points in $\mathcal{D}$ are offset by $\|q\|_2^2$, the ranking between points does not change. Hence, the transformation suffices to answer any ℓ_2 norm based RAS query using a transformed RAR query with rank set to 1. A visual representation of this lifting transformation is shown in Figure 2. $\square$

D.2 Cosine Distance: Proof of Lemma 2

PROOF. For a query q , the cosine distance is given by,

$$\mathbf{d}(p, q) = 1 - \frac{q^\top p}{\|q\|_2 \cdot \|p\|_2}. \quad (7)$$

The numerator of the Equation 7, $q^\top p$, can be interpreted as a dot product between the linear function q and p . Therefore,

$$\begin{aligned} \mathbf{d}(p, q) &= 1 - \left(\frac{q}{\|q\|_2} \right)^\top \frac{p}{\|p\|_2} \\ &= 1 + \sum_{j=1}^d \frac{-q[j]}{\|q\|_2} \frac{p[j]}{\|p\|_2} \end{aligned}$$

Removing the constant term from the scoring function, the ranking generated by Equation 8 is the same as Equation 7.

$$\text{score}(p) = \sum_{j=1}^d \frac{-q[j]}{\|q\|_2} \frac{p}{\|p\|_2} \quad (8)$$

Therefore, transforming each data point p to $\frac{p}{\|p\|_2}$ in the transformed space, given a query $q \in \mathbb{R}^d$, the corresponding weight vector of the scoring function is $f = (\frac{q[1]}{\|q\|_2}, \dots, \frac{q[d]}{\|q\|_2})$. $\square$

E Our Algorithm Based on k -th Level of Arrangement

In this section, we describe the details of KTHLEVEL algorithm. We begin by introducing key concepts from computational geometry, and then describe our algorithm in detail.

E.1 Preliminaries

Duality. We use the classic dual space transformation that maps points to hyperplanes and vice versa. This transformation is well-defined and preserves intersection relationships [29, 41]. Formally, the dual of a point $p \in \mathbb{R}^d$ is defined as:

$$\text{dual}(p) := \{x \in \mathbb{R}^d \mid x^\top p = 1\}.$$

In other words, the transformation $\text{dual}(\cdot)$ maps a point in the *primal space* to a hyperplane in the *dual space*. This hyperplane is represented by the equation:

$$h : \sum_{i=1}^d p[i] \cdot x_i = 1.$$

We refer to the original input domain as the *primal space*, and the space of dual hyperplanes as the *dual space*. Figure 11 provides a visual illustration of this transformation. The left figure shows the primal space, while the middle figure shows the dual hyperplanes in the dual space. The following key observation follows from this duality. A formal proof can be found in [41].

LEMMA 7. *Let f be a scoring function (i.e., a direction vector), and let $p^{(1)}, \dots, p^{(n)}$ be the points sorted in descending order according to their score under f , so that $f^\top p^{(1)} > f^\top p^{(2)} > \dots$. Then, for any $a < b$, the intersection point of $\text{dual}(p^{(b)})$ with the ray in direction f lies closer to the origin than that of $\text{dual}(p^{(a)})$.* $\square$

For instance, in Figure 11, the point B has the highest score under f_1 , and thus its dual $\text{dual}(B)$ intersects the ray f_1 in the dual space earlier (closer to origin) than any other point's dual.

E.1.1 k -th Level of Arrangement. Consider the set of dual hyperplanes $\mathcal{H} = \{\text{dual}(p) \mid p \in \mathcal{D}\}$. $\mathcal{H}$ defines a dissection of $\mathbb{R}^d$ into connected convex cells, called the *arrangement* of the hyperplanes [41].

To simplify the explanations, let $d = 2$. Sweep a ray f counterclockwise from the x -axis to the y -axis. At each orientation of f , the ray intersects all dual hyperplanes (dual lines) in $\mathcal{H}$. These intersection points can be sorted by their distance from the origin. The k -th level is the locus of points that lie on the k -th closest dual line intersected by f , as the direction of f varies continuously [41].

In general d -dimensional space, consider a ray f originating at the origin and sweeping over all directions on the unit sphere $\mathbb{S}^{d-1}$.

For each direction f , the ray intersects the hyperplanes in $\mathcal{H}$ in up to n distinct points, which can again be ordered by increasing distance from the origin. The k -th level is the set of all such k -th intersection points across all directions $f \in \mathbb{S}^{d-1}$.

Figure 11 illustrates the first and second levels of the arrangement in $\mathbb{R}^2$, shown in orange and green, respectively.

E.2 KTHLEVEL Algorithm

In this subsection, we formally present our algorithm, KTHLEVEL. We begin by describing the *preprocessing* phase, followed by a discussion of the *query phase*, and conclude with a theoretical analysis of the algorithm's performance. The overall structure of the algorithm applies uniformly across dimensions $d \geq 2$; however, for the purpose of analysis, we distinguish between the planar case ($d = 2$) and the general higher-dimensional case ($d > 2$).

Preprocessing. During the preprocessing phase, we construct the arrangement of the dual hyperplanes induced by the point set $\mathcal{D}$, and the levels of the arrangement for all $1 \leq k \leq n$. We assume access to an oracle $\mathcal{K}$ that, for a given i , returns an efficient data structure representing the i -th level of the arrangement [41]. By calling this oracle for each $i \in [n]$, we obtain the complete set of structures needed for query processing. Each such structure allows constant-time access to the hyperplanes intersecting on each face of the arrangement, as well as the points in the primal space corresponding to those hyperplanes [41].

Query Phase. In the query phase, we are given a scoring function f and a rank i , and the goal is to retrieve the point $p^{(i)}$, which is the i -th highest-ranked point with respect to f . By Lemma 7, the intersection of the i -th level of arrangement of dual hyperplanes with f corresponds with the desired point $p^{(i)}$. Algorithm 1 shows the pseudo-code of our process for finding this intersection.

Consider the 3rd level of the arrangement highlighted (in orange) in the Figure 11 (right figure). In 2D, the k -th level is a non-convex chain of line segments, each belonging to a dual line. Now, let the query function be f_2 , while $i = 3$. Our objective is to find $p^{(3)} = \text{rank}_{f_2}^{-1}(3)$. The line segment in the 3rd level that intersects with the vector of f_2 is $\text{dual}(D)$. Hence, the point $p^{(3)} = D$ in this example. Note that applying a binary search on the endpoints of the line segments of the 3rd level, one can find the intersection of any function f with any level i in a time logarithmic to the number of line segments.

From the above example, observe that the k -th level is a chain of line segments. Similarly, for a general $d \geq 2$, the k -th level is a chain of $(d-1)$ -faces. A d' -face of an arrangement is a d' -dimensional convex cell, formed by the intersection of $d-d'$ hyperplanes. For example, 0-faces are vertices (points), 1-faces are edges (segments or lines), and so on.

Each function f , in such cases, is an origin-anchored vector that intersects one of the faces of each level $i, \forall i \in [n]$. Projecting the endpoints of these faces on the surface of the unit sphere $\mathbb{S}^{d-1}$ forms an arrangement in the $(d-1)$ -dimensional space. For example, when $d = 3$, each k -th level is a chain of plane segments, and their projection onto the unit ball (centered at the origin) is an arrangement of lines.

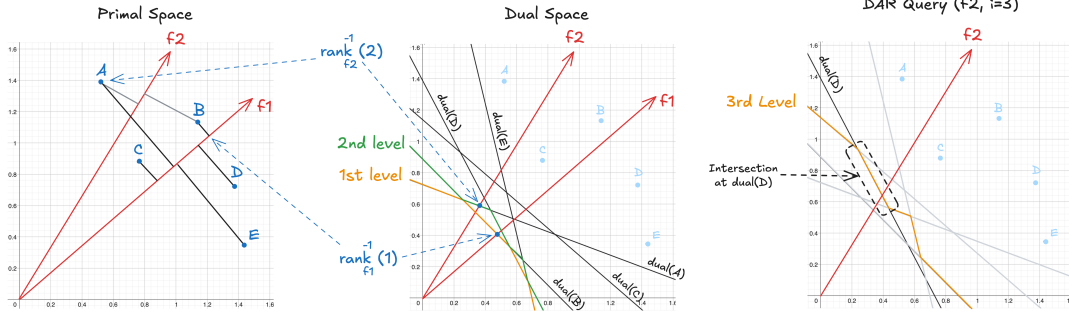


Figure 11: A 2D representation of dual space and the k -th level of arrangements (For simplicity, only the first quadrant is presented). The right figure shows an example of running KthLevel for $i = 3$. The result of this query is the point D .

Algorithm 1 KTHLEVEL-QUERY(f, i)

Input: The scoring function f and the order i

Output: The i -th highest scored point according to f : $p^{(i)}$

- 1: $Level_i \leftarrow$ preprocessed data structure ▷ Get the i -th level
 - 2: $face \leftarrow$ Intersection($f, Level_i$) ▷ Get the intersection
 - 3: $h \leftarrow$ The hyperplane corresponding to $face$ in arrangement
 - 4: $p^{(i)} \leftarrow dual^{-1}(h)$
 - 5: **return** $p^{(i)}$
-

Now, consider the intersection point of the query function f with the unit sphere $\mathbb{S}^{d-1}$. Finding the face segment of a level i with a function f is equivalent to finding the cell of the arrangement on the surface of the sphere that intersects with f . This problem is known as *point location* on the arrangement of hyperplanes [73]. Line 2 of Algorithm 1 uses a state-of-art point-location algorithm for finding the intersection of f with the i -th level.

E.3 Analysis of KthLevel Algorithm

Preprocessing. In 2D, the complexity of the k -th level in an arrangement of n lines in the plane is $O(nk^{1/3})$ [38]. Hence, considering an arbitrary level $i = O(n)$, the complexity of the i -th level is $O(n^{4/3})$. The total preprocessing time to construct all levels of the arrangement is $O(n^2)$, where each individual level can be built in $O(n \log n + nk)$ time [43]. In 3D, the complexity of the k -th level is $O(nk^{3/2})$ [87]. As a result, the complexity of each level $i \in [n]$ is bounded by $O(n^{5/2})$. The complexity of the arrangement in the general d -dimensional setting is $O(n^d)$, and the total time required to construct it is also $O(n^d)$ [3, 41].

Query Time. During the query time, Algorithm 1 finds the intersection of the i -th level with the vector of the function f . Given that the total number of faces in each level $i \in [n]$ is bounded by $O(n^d)$, identifying this intersection using a state-of-the-art algorithm for point location on the arrangement of hyperplanes is in $O(\log n)$, ignoring the constant factors that depend on d [73].

F Analysis of the Eps2D Algorithm

We analyze the time and space complexity of the Eps2D algorithm (discussed in section 6.1) step-by-step.

(Step i) Finding the i_ℓ -th and i_u -th point in $\mathcal{N}_\epsilon$ of size m takes $O(m)$ using the extension of the median finding algorithm.

(Step ii) This step involves solving a simplex range searching query in 2D (with stripe ranges). Using the Matoušek's efficient range searching technique [71], this can be done in $O(\sqrt{n} + |\mathcal{D}_o|)$ time with $O(n)$ space.

(Step iii) We perform a half-space range counting query. Again using Matoušek's partition tree [71], this takes $O(\sqrt{n})$ time and $O(n)$ space. (Step iv) Finally, we sort the set $\mathcal{D}_o$, which requires $O(|\mathcal{D}_o| \log |\mathcal{D}_o|)$ time.

Space Complexity. All operations use data structures with linear space requirements. Thus, the total space usage is linear in input.

Query Time Complexity. To analyze the runtime precisely, we bound the size of the result set $\mathcal{D}_o$ as follows:

LEMMA 8. The size of the result set $\mathcal{D}_o$ is $O(\epsilon n)$. □

PROOF. The result set $\mathcal{D}_o$ consists of the points in the dataset $\mathcal{D}$ that lie between the i_ℓ -th and i_u -th ranked points in the sample $\mathcal{N}_\epsilon$. Define the corresponding score interval as the stripe range:

$$S = \{x \in \mathbb{R}^d \mid f^\top q_\ell \leq f^\top x \leq f^\top q_u\}.$$

Then, $|\mathcal{D}_o| = |S \cap \mathcal{D}|$. Since $\mathcal{N}_\epsilon$ is an ϵ -sample of $\mathcal{D}$ for stripe ranges, we have (with high probability): $\left| \frac{|S \cap \mathcal{N}_\epsilon|}{|\mathcal{N}_\epsilon|} - \frac{|\mathcal{D}_o|}{|\mathcal{D}|} \right| \leq \epsilon$. From Equation 1, we know that $|S \cap \mathcal{N}_\epsilon| = |i_u - i_\ell| \leq 2m\epsilon + 2$. Hence:

$$\left| \frac{|S \cap \mathcal{N}_\epsilon|}{m} - \frac{|\mathcal{D}_o|}{n} \right| \leq \epsilon \implies \frac{|\mathcal{D}_o|}{n} \leq \frac{2m\epsilon + 2}{n} + \epsilon \implies |\mathcal{D}_o| = O(\epsilon n).$$

□

In 2D, the size of the ϵ -sample satisfies (see Section 4): $m = O\left(\frac{1}{\epsilon^2} \log \frac{1}{\epsilon}\right)$. Combining all steps, the total runtime is:

$$T(\text{Eps2D}) = O\left(m + \sqrt{n} + |\mathcal{D}_o| \log |\mathcal{D}_o|\right).$$

Using $|\mathcal{D}_o| = O(\epsilon n)$ and choosing $\epsilon = n^{-1/3}$ (which minimizes the overall runtime), we get:

$$T(\text{Eps2D}) = O\left(n^{2/3} \log n\right).$$

G Missing Proofs

G.1 Proof of Lemma 4

PROOF. Let $\mathcal{H}$ be the half-space defined by: $\mathcal{H} = \{x \in \mathbb{R}^d \mid f^\top x \geq f^\top p^{(i)}\}$. This half-space is perpendicular to the direction vector f and contains exactly the top i points of $\mathcal{D}$ ranked by f , including $p^{(i)}$. In other words, $|\mathcal{H} \cap \mathcal{D}| = i$.

Since $\mathcal{N}_\epsilon$ is an ϵ -sample of $\mathcal{D}$ for half-space (and slab) ranges, it preserves the proportion of points in any half-space up to additive error ϵ . Therefore,

$$\left| \frac{|\mathcal{H} \cap \mathcal{N}_\epsilon|}{|\mathcal{N}_\epsilon|} - \frac{|\mathcal{H} \cap \mathcal{D}|}{|\mathcal{D}|} \right| \leq \epsilon.$$

Substituting $|\mathcal{H} \cap \mathcal{D}| = i$, and denoting $n = |\mathcal{D}|$ and $m = |\mathcal{N}_\epsilon|$, we obtain:

$$\left| \frac{|\mathcal{H} \cap \mathcal{N}_\epsilon|}{m} - \frac{i}{n} \right| \leq \epsilon,$$

which implies:

$$m \left(\frac{i}{n} - \epsilon \right) \leq |\mathcal{H} \cap \mathcal{N}_\epsilon| \leq m \left(\frac{i}{n} + \epsilon \right).$$

Rounding to integers, define:

$$i_\ell = \left\lfloor m \left(\frac{i}{n} - \epsilon \right) \right\rfloor, \quad i_u = \left\lceil m \left(\frac{i}{n} + \epsilon \right) \right\rceil.$$

These bounds indicate that the number of points in $\mathcal{N}_\epsilon$ with score at least $\text{score}_f(p^{(i)})$ lies between i_ℓ and i_u . Equivalently, the score $\text{score}_f(p^{(i)})$ lies between the scores of the i_u -th and i_ℓ -th ranked points in $\mathcal{N}_\epsilon$. We conclude that:

$$\text{score}_f(q_\ell) \leq \text{score}_f(p^{(i)}) \leq \text{score}_f(q_u),$$

as desired. $\square$

H Dynamic Setting

In this section, we discuss the dynamic setting. We consider two types of dynamic operations: **Insert(p)** and **Delete(p)**. For the KTHLEVEL algorithm, these operations are supported following the literature on dynamically maintaining levels of arrangements [4, 24]. Below, we focus on the EPSHIER algorithm; analogous results can be shown for EPSRANGE.

Insert(p). This operation adds a new point p to the dataset $\mathcal{D}$. For EPSHIER, we must update both the ϵ -sample $\mathcal{N}_\epsilon$ and the hierarchical sampling structure described in Section 6.3. Following Theorem 3, we want to maintain an unbiased random subset of size $|\mathcal{N}_\epsilon|$ of $\mathcal{D}$ as an ϵ -sample. Our goal is to maintain such a random subset without resampling the entire dataset. This can be efficiently achieved using *reservoir sampling* [96], which allows us to maintain a uniform random sample as new elements arrive. To update the hierarchical structure used for the SRR problem, we apply this reservoir sampling procedure bottom-up, from the base layer to higher layers. At each layer, we decide whether to include the new point, as each layer is a random sample of the one below it. This process takes $O(\log n)$ time since the index contains $\log n$ layers.

Delete(p). This operation removes p from $\mathcal{D}$. Under the *backing sample* method [47], if $p \in \mathcal{N}_\epsilon$, we delete it from the sample. If the sample size drops below a predefined lower bound L , we *rebuild* $\mathcal{N}_\epsilon$ by resampling all the points. The amortized time for this operation (based on the definition of L) is $O(1)$ [47]. For the hierarchical structure, each layer ℓ is maintained similarly, with its own bound L_ℓ , and each layer ℓ is the sample maintained at level $\ell-1$. Hence, a rebuild at level ℓ only scans the immediately lower level, not the entire dataset, keeping the amortized cost low. In other words, this rescanning at layer ℓ takes $O(|\mathcal{L}_{\ell-1}|) = O(\frac{n}{2^{\ell-1}})$, but the amortized is still $O(1)$ for this layer and it takes $O(\log n)$ to resample all the top layers. As a result, the total amortized time for this operation is $O(\log n)$ for the hierarchical structure.

I Analysis of the EPSRANGE Algorithm

The analysis is similar to EPS2D, however, the time complexity of the algorithm depends on the specific range counting and simplex range searching we use at steps ii and iii.

Step i takes $O(m)$ for finding the thresholds in the ϵ -sample. Step ii requires solving the SRR problem. Following the simplex range searching algorithms, we can solve this problem in $O(n^{1-1/d} + |\mathcal{D}_o|)$ with $O(n)$ space [72]. To report the exact response to RAR, i.e., $p^{(i)}$, we proceed to steps iii and iv. In step iii, we solve an instance of range counting in d dimensions, which takes $O(n^{1-1/d})$ [72] time and linear space. Finally, in step iv, we sort $\mathcal{D}_o$, which takes $O(|\mathcal{D}_o| \log |\mathcal{D}_o|)$.

Space Complexity. The space complexity of this algorithm is linear to the input size, since all structures require linear space.

Time Complexity for the κ -RAR problem. The total query time of the algorithm for finding the conformal set is:

$$T(\text{EPSRANGE}_{\kappa\text{-RAR}}) = O(m + n^{1-1/d})$$

The size of ϵ -sample m is $O(\frac{d}{\epsilon^2} \log \frac{d}{\epsilon})$, since the VC-dimension of stripe range is $\delta = d + 1$ [50, 51]. This gives us the following query time:

$$T(\text{EPSRANGE}_{\kappa\text{-RAR}}) = O(\max\{n^{1-1/d}, \frac{d}{\epsilon^2} \log \frac{d}{\epsilon}\}) = \Omega(n^{1-1/d}).$$

The output is a set $\mathcal{D}_o$ with size $O(\epsilon n)$, guaranteed to contain $p^{(i)}$ (see Lemma 8). Note that the final time also depends on the choice of parameter ϵ , which determines the size of output $\mathcal{D}_o$.

Time Complexity for the RAR problem. The exact version also includes the steps iii and iv:

$$T(\text{EPSRANGE}_{\text{Exact}}) = O(m + n^{1-1/d} + |\mathcal{D}_o| \log |\mathcal{D}_o|).$$

Based on Lemma 8, we have $|\mathcal{D}_o| = O(\epsilon n)$. As a result, by choosing $\epsilon = n^{-1/3}$, we get the following runtime:

$$T(\text{EPSRANGE}_{\text{Exact}}) = O(\max\{n^{1-1/d}, dn^{2/3} \log n\})$$

THEOREM 9. *The EPSRANGE algorithm solves the relaxed version of RAR problem with linear space usage and $O(\max\{n^{1-1/d}, \frac{d}{\epsilon^2} \log \frac{d}{\epsilon}\})$ time. The exact version can also be solved with the same space usage and $O(\max\{n^{1-1/d}, dn^{2/3} \log n\})$ time.*

J Analysis of Hierarchical Sampling

The total space usage for storing the hierarchical layers is linear to the input, since the size of each layer decreases exponentially:

$$\sum_{\ell \leq L} |\mathcal{L}_\ell| = \sum_{\ell \leq L} \frac{n}{r^\ell} = O(n),$$

assuming $r = O(1)$. The runtime of Algorithm 3 is dominated by the **for** loop at line 6, where the layers are constructed, and edges and area sets are updated. At each layer ℓ , we sample $|\mathcal{L}_\ell|$ points. Then, for each point in $\mathcal{L}_{\ell-1}$, we compute its nearest neighbor in $\mathcal{L}_\ell$. The time complexity at the layer ℓ is:

$$T_\ell = O(d \cdot |\mathcal{L}_\ell| \cdot |\mathcal{L}_{\ell-1}|).$$

Summing over all layers, the total runtime becomes:

$$\sum_\ell T_\ell = \sum_\ell d \cdot \frac{n}{r^\ell} \cdot \frac{n}{r^{\ell-1}} = \sum_\ell \frac{dn^2}{r^{2\ell-1}} = O(dn^2),$$

again assuming $r = O(1)$. For computing the smallest enclosing ball of each area, we use Welzl's algorithm, which runs in linear time with respect to the number of points [98].

THEOREM 10. *The preprocessing phase of the hierarchical sampling algorithm takes $O(dn^2)$ time and uses linear space.*

K Additional Pseudo-codes

Algorithm 2 EPS2D-QUERY($\mathcal{N}_\varepsilon, \mathcal{D}, f, i$)

Input: Precomputed ε -sample $\mathcal{N}_\varepsilon$ of size m , original dataset $\mathcal{D}$, scoring vector f , target rank i

Output: The exact k -th ranked point $p^{(i)}$ in $\mathcal{D}$ under score_f

- 1: Calculate i_ℓ and i_u ▷ equation 1
 - 2: $q_\ell \leftarrow \text{rank}_{\mathcal{N}_{\varepsilon,f}}^{-1}(i_\ell)$, $q_u \leftarrow \text{rank}_{\mathcal{N}_{\varepsilon,f}}^{-1}(i_u)$ ▷ Median of medians algorithm
 - 3: $\ell \leftarrow \text{score}_f(q_\ell)$, $u \leftarrow \text{score}_f(q_u)$ ▷ end of step i
 - 4: $\mathcal{D}_o \leftarrow \mathcal{S}_{f,\ell,u} \cap \mathcal{D}$ ▷ via SRS query; step ii
 - 5: $|H_u| \leftarrow |\{p \in \mathcal{D} \mid f^\top p \geq u\}|$ ▷ via range counting; step iii
 - 6: Sort $\mathcal{D}_o$ ascending by score_f
 - 7: **return** the $(i - |H_u|)$ -th point in the sorted list ▷ step iv
-

Algorithm 3 HIERARCHICAL-SAMPLING-PREPROCESS($\mathcal{D}, r$)

Input: The dataset $\mathcal{D}$ and the exponential decay rate r

Output: The hierarchical graph $\mathcal{G}(\mathcal{D})$ which contains a list of layers $\mathcal{L}$, neighboring relations N , and enclosing balls of nodes $\mathcal{B}$.

- 1: $L \leftarrow \lfloor \log_r n \rfloor$ ▷ Number of layers
 - 2: $\mathcal{L}_\ell \leftarrow []$, $\forall \ell \leq L$ ▷ Placeholder for the layers
 - 3: $\mathcal{L}_0 \leftarrow \mathcal{D}$ ▷ Base layer
 - 4: $\mathcal{A}_0(p) = \{p\}$, $\forall p \in \mathcal{L}_0$ ▷ Area of nodes
 - 5: $N_0(p) \leftarrow \emptyset$, $\forall p \in \mathcal{L}_0$ ▷ Neighbors
 - 6: **for** layer $\ell = 1$ to L **do**
 - 7: $\mathcal{L}_\ell \leftarrow \text{Random sample of size } \frac{|\mathcal{L}_{\ell-1}|}{r} \text{ from } \mathcal{L}_{\ell-1}$
 - 8: $\mathcal{A}_\ell(p) \leftarrow \emptyset$, $\forall p \in \mathcal{L}_\ell$
 - 9: $N_\ell(p) \leftarrow \emptyset$, $\forall p \in \mathcal{L}_\ell$
 - 10: **for** point $p \in \mathcal{L}_{\ell-1}$ **do**
 - 11: $c^* \leftarrow \arg \min_{c \in \mathcal{L}_\ell} \text{Dist}(p, c)$ ▷ Equation 4
 - 12: $N_\ell(c^*) \leftarrow N_\ell(c^*) \cup \{p\}$ ▷ Add neighbor
 - 13: $\mathcal{A}_\ell(c^*) \leftarrow \mathcal{A}_\ell(c^*) \cup \mathcal{A}_{\ell-1}(p)$ ▷ Update area
 - 14: $\mathcal{B}_\ell(p) \leftarrow \text{EnclosedBall}(\mathcal{A}_\ell(p))$, $\forall \ell \leq L, p \in \mathcal{L}_\ell$
 - 15: $\mathcal{L} \leftarrow \{\mathcal{L}_\ell \mid \ell \leq L\}$
 - 16: $N \leftarrow \{N_\ell(p) \mid \forall \ell \leq L, p \in \mathcal{L}_\ell\}$
 - 17: $\mathcal{B} \leftarrow \{\mathcal{B}_\ell(p) \mid \forall \ell \leq L, p \in \mathcal{L}_\ell\}$
 - 18: **return** $(\mathcal{L}, N, \mathcal{B})$ ▷ layers, edges, and enclosing balls
-

Algorithm 4 HIERARCHICAL-SAMPLING-QUERY($\mathcal{G}(\mathcal{D}), \mathcal{S}_{f,\ell,u}$)

Input: The preprocessed graph $\mathcal{G}(\mathcal{D})$ and the query stripe range $\mathcal{S}_{f,\ell,u}$.

Output: The set of points $\mathcal{D}_o = \mathcal{S}_{f,\ell,u} \cap \mathcal{D}$.

- 1: $\mathcal{D}_o \leftarrow \emptyset$ ▷ Placeholder for result
 - 2: $\text{candidates} \leftarrow \mathcal{L}_L$
 - 3: **for** curr_layer from L to 1 **do** ▷ top-down
 - 4: $\text{tmp} \leftarrow \emptyset$ ▷ Placeholder for new candidates
 - 5: **for** each point $p \in \text{Candidates}$ **do**
 - 6: **if** $\mathcal{B}_{\text{curr_layer}}(p) \cap \mathcal{S}_{f,\ell,u} \neq \emptyset$ **then**
 - 7: $\text{tmp} \leftarrow \text{tmp} \cup \mathcal{N}_{\text{curr_layer}}(p)$ ▷ Explore the area
 - 8: $\text{candidates} \leftarrow \text{tmp}$
 - 9: **for** $p \in \text{candidates}$ **do**
 - 10: **if** $p \in \mathcal{S}_{f,\ell,u}$ **then**
 - 11: $\mathcal{D}_o \leftarrow \mathcal{D}_o \cup \{p\}$
 - 12: **return** $\mathcal{D}_o$
-

L Lower Bounds and Optimality of Our Algorithms

In this section, we analyze the complexity of the RAR problem and show the near-optimality of our proposed algorithms. Specifically, the well-studied *half-space range counting* problem [2, 26] reduces to our problem. Hence, the complexity of the state-of-the-art algorithms for half-space range counting provides a lower bound for the RAR problem.

The half-space range counting problem is defined as follows: given a point set in $\mathbb{R}^d$ and a query half-space H , determine the number of points lying within H .

LEMMA 11. *Let $\mathcal{A}$ be an algorithm that solves the RAR problem with query time $T(\mathcal{A})$ and space complexity $S(\mathcal{A})$. Then, using $\mathcal{A}$, one can solve an instance of the half-space range counting problem using space $S(\mathcal{A})$ and query time $\Theta(\log n \cdot T(\mathcal{A}))$.*

PROOF. To count the number of points within a given half-space H , we can perform a binary search over $k \in \{1, 2, \dots, n\}$. In each iteration, we query $\mathcal{A}$ to retrieve the k -th ranked point $p^{(k)}$. If $p^{(k)} \in H$, then the count of points in H is at least k ; otherwise, it is less. This process requires $\Theta(\log n)$ queries to $\mathcal{A}$, resulting in total query time $\Theta(\log n \cdot T(\mathcal{A}))$. $\square$

It is known that the query time for half-space range counting in $\mathbb{R}^d$ with m units of space satisfies the following lower bound [26]:

$$\Omega\left(\frac{n}{m^{1/d} \log n}\right). \quad (9)$$

Applying Lemma 11, this implies a lower bound for the RAR problem:

$$T(\mathcal{A}) \cdot \log n \geq \frac{n}{m^{1/d} \log n} \implies T(\mathcal{A}) \geq \frac{n}{m^{1/d} \log^2 n}.$$

Now, these are the two important observations:

Observation 1. If the algorithm uses linear space, i.e., $m = O(n)$, then the query time cannot be faster than $O(n^{1-\frac{1}{d}})$ up to logarithmic factors.

Observation 2. If the algorithm uses exponential space, i.e., $m = 2^{\Theta(d)}$, then the lower bound becomes $\Omega(\log n)$. In fact, our exact algorithm **KTHLEVEL** has logarithmic query time but requires exponential space, which matches this bound. Note that querying **KTHLEVEL** n times is equivalent to sorting all n elements, which takes $\Omega(n \log n)$ time.

Conclusion. These results demonstrate that our algorithms for the RAR problem are optimal up to logarithmic factors, given the space-time trade-offs imposed by known lower bounds.

M More on Experiments

M.1 Dataset Details

We use both synthetic and real-world datasets for our evaluation. The synthetic datasets are generated using the Zipfian distribution, with variations in dimensionality, dataset size, and distribution parameters. The Zipfian distribution is particularly challenging for range searching due to its inherent skewness and is commonly observed in many real-world scenarios. It allows us to evaluate the algorithms under non-uniform workloads.

For our real-world experiments, we use three datasets: the US Used Cars dataset [75], the FIFA 2023 dataset [63], and the US Flights dataset [94]. The US Used Cars dataset contains approximately 3 million entries with 66 attributes describing vehicle specifications, market prices, and regional features, making it suitable for evaluating ranking and filtering performance over high-dimensional numeric attributes. The FIFA 2023 dataset comprises around 300,000 records with 54 player and team attributes, including overall ratings, skills, and physical characteristics, and provides a structured benchmark for similarity and top- k retrieval tasks. Finally, the US Flights dataset contains over 5 million flight records with 31 attributes collected from the U.S. Department of Transportation, including flight times, delays, cancellations, and carrier information, offering a real-world testbed with temporal and categorical dimensions for large-scale ranked retrieval evaluation.

Scoring Functions. In our experiments, we generate a uniformly random unit vector f as the ranking function on all the above datasets.

Similarity Search Datasets. For the similarity search task, we evaluate our methods on five standard benchmark datasets from ann-benchmarks.com, covering both vision and language domains:

- **SIFT-128-EUCLIDEAN** [58]: Contains 1M vectors in a 128-dimensional space with ℓ_2 (Euclidean) distance, derived from local image descriptors and widely used as a canonical benchmark for large-scale visual similarity search.
- **FASHION-MNIST-784-EUCLIDEAN** [99]: Contains 60K vectors in a 784-dimensional space with ℓ_2 distance, obtained from grayscale images of fashion items and representing a moderate-scale, high-dimensional vision dataset.
- **GIST-960-EUCLIDEAN** [58]: Contains 1M vectors in a 960-dimensional space with ℓ_2 distance, based on global image descriptors that capture coarse scene-level structure, making it significantly higher-dimensional than SIFT.
- **GLOVE-100-ANGULAR** and **GLOVE-25-ANGULAR**: Contain 1M word embedding vectors in 100- and 25-dimensional spaces, respectively, using cosine similarity, and serve as standard benchmarks for semantic similarity search in natural language processing.

M.2 More on Stripe Range Retrieval

Preprocessing (Indexing) Phase. During the preprocessing phase in all the baselines, an index structure is constructed. Figure 12 (left) shows the memory usage of these indices as a function of the dataset size n , while the right plot illustrates the corresponding preprocessing time. As shown, the space overhead of Hierarchical Sampling is comparable to that of KD-Tree and R-Tree, both of which are commonly used in existing systems. Moreover, the space usage grows linearly with the dataset size n . The time required to build the Hierarchical Sampling index is also on par with that of the Partition Tree and R-Tree.

More Comparison Reports. Figure 13 shows the query time of different methods, when the value of d is 4 and 32.

Figure 14 shows the experiments for Stripe Range Searching on FIFA dataset.

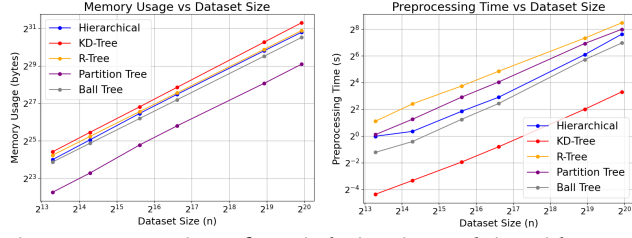


Figure 12: Comparison of SRR indexing time and size with respect

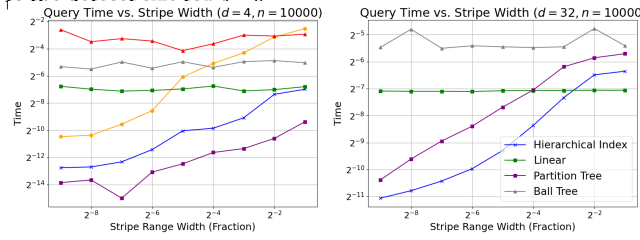


Figure 13: Comparison of SRR query time with respect to the stripe width across different dimensionalities on the synthetic data.

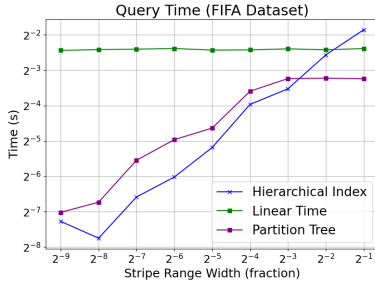
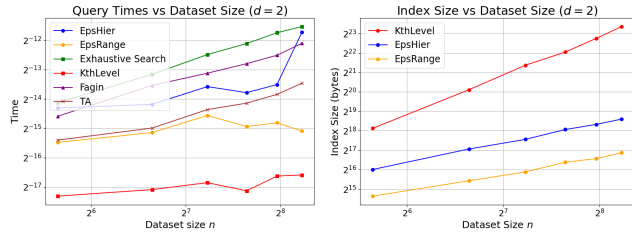


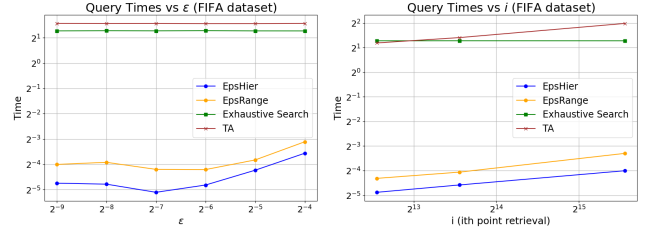
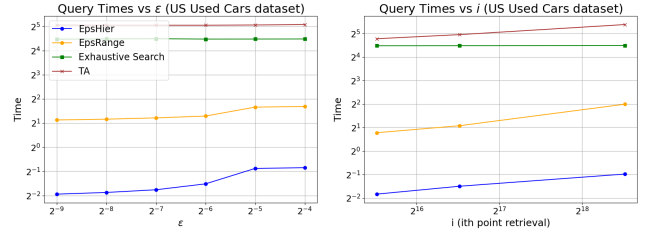
Figure 14: Comparison of the query time of Hierarchical Sampling on SRR problem vs the stripe range width. The scoring functions are sampled uniformly at random from a hypersphere.

M.3 More on Direct-Access and Conformal Set Retrieval

KthLevel Algorithm. Figure 15 shows the result of running the KthLevel algorithm on smaller datasets in 2D. As shown in, the KthLevel algorithm achieves substantial speedups. However, this improvement comes at the cost of increased index size.

Figure 15: Result of applying KthLevel for solving RAR problem in 2D. Generally, KthLevel does not scale well with dimension d and size n . Here, EpsRange in 2D is equivalent to Eps2D algorithm.

Other Real Datasets. Figures 16 and 17 compare the performance of the algorithms on the US Used Cars and FIFA datasets, where the parameters ϵ and rank i are varied across different runs.

Figure 16: Comparing the query time of EpsHier and EpsRange on κ -RAR problem vs value of ϵ and rank i on FIFA 2023 dataset.Figure 17: Comparing the query time on κ -RAR problem vs value of ϵ and rank i on US Used Cars Dataset.

M.4 Random-Access Similarity Search

Adapting ANN Indexes to the κ -RAR Problem.

In this section, we describe how standard approximate nearest neighbor (ANN) indexes—such as HNSW and Product Quantization—are adapted to the random-access similarity search setting, and used to compare their performance with our proposed algorithm, EpsHIER, in Section 7.3.

Our algorithm directly solves the κ -RAR problem with $\kappa = \epsilon n$ for a given ϵ . The output is a set of size κ . To evaluate recall, we check whether the ground-truth i -th point, denoted by $p^{(i)}$, is contained in this output set (i.e., the conformal set).

In contrast, ANN indexes are designed to return the top- k nearest neighbors. To approximate the i -th ranked point, we query each ANN index with $k = i + 10$, where the additional constant margin ensures that the true i -th neighbor is covered in the best-case scenario. Depending on the index configuration, the output size may be $O(k)$ for large k , or bounded by a maximum size M . For example, in HNSW, M corresponds to the beam search candidate set size. Consequently, the final output size is $\min(M, k)$.

Since this output does not directly correspond to a solution of the κ -RAR problem, we apply a post-processing step to ensure a fair comparison. Specifically, we select the last ϵn points (i.e., those with the largest distances) from the ANN output and then check whether $p^{(i)}$ lies within this subset.

More Results on the RAS Task.

Table 2 reports the recall and query time of ANN indexes across all five benchmark datasets for varying values of i , ranging from small values ($i = O(1)$) to the median and third quartile of the dataset.

We observe that the performance of all ANN indexes degrades significantly as i grows beyond $O(1)$. In particular, for large values of i , these methods fail to retrieve the required point, resulting in near-zero recall. Moreover, their query time increases substantially, as the algorithms must iterate over an increasing number of candidates and compute distances for up to i points.

In contrast, our algorithm, **EpsHier**, exhibits stable performance across all values of i , achieving fast query times and a guaranteed recall of 100%. This guarantee holds with high probability, conditioned on the sampled set forming an ε -sample, as discussed in Section 6.

Figure 18 compares the indexing (preprocessing) time and index size of **EpsHier** with those of the ANN indexes used in this experiment. We observe that both the index size and preprocessing time of **EpsHier** are comparable to standard ANN methods. However, unlike ANN indexes, which do not meaningfully solve the RAS

problem and exhibit zero recall for large i , our approach provides provable correctness guarantees across the entire range of i .

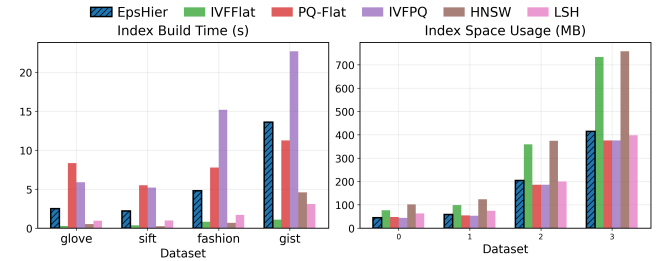


Figure 18: Comparing the space usage and building (indexing) time for Random-Access Similarity Search task

Method	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall	Time (s)	Recall
fashion-mnist-784-euclidean ($n = 60,000$)																
	$i = 10$		$i = 29 (n/2^{11})$		$i = 117 (n/2^9)$		$i = 468 (n/2^7)$		$i = 1875 (n/2^5)$		$i = 7500 (n/2^3)$		$i = 30000 (n/2^1)$		$i = 45000 (3n/4)$	
EpsHier* (Ours)	0.018	1.000	0.021	1.000	0.021	1.000	0.020	1.000	0.022	1.000	0.021	1.000	0.019	0.920	0.023	1.000
Quick Select	0.114	1.000	0.113	1.000	0.112	1.000	0.111	1.000	0.111	1.000	0.113	1.000	0.117	1.000	0.117	1.000
IVFFlat	0.003	1.000	0.003	1.000	0.003	1.000	0.004	1.000	0.007	1.000	0.018	0.000	0.065	0.000	0.069	0.000
PQ	0.003	0.920	0.003	0.790	0.004	0.670	0.005	0.470	0.010	0.530	0.028	0.000	0.080	0.000	0.111	0.000
IVFPQ	0.001	0.810	0.001	0.610	0.001	0.400	0.002	0.450	0.007	0.420	0.026	0.000	0.030	0.000	0.032	0.000
HNSW	0.000	1.000	0.000	0.990	0.001	0.880	0.002	0.330	0.002	0.050	0.002	0.000	0.005	0.000	0.006	0.000
LSH	0.053	0.390	0.060	0.270	0.060	0.250	0.059	0.210	0.056	0.270	0.041	0.030	0.085	0.000	0.116	0.000
glove-100-angular ($n = 1,183,514$)																
	$i = 10$		$i = 144 (n/2^{13})$		$i = 2311 (n/2^9)$		$i = 9246 (n/2^7)$		$i = 36984 (n/2^5)$		$i = 147939 (n/2^3)$		$i = 591757 (n/2^1)$		$i = 887635 (3n/4)$	
EpsHier* (Ours)	0.022	1.000	0.026	1.000	0.027	1.000	0.023	1.000	0.021	1.000	0.022	1.000	0.022	1.000	0.022	1.000
Quick Select	3.494	1.000	3.386	1.000	3.393	1.000	3.389	1.000	3.422	1.000	3.502	1.000	3.606	1.000	3.616	1.000
IVFFlat	0.006	0.990	0.006	0.980	0.011	0.960	0.027	0.950	0.089	0.840	0.339	0.000	1.261	0.000	1.352	0.000
PQ	0.044	1.000	0.044	0.920	0.050	0.600	0.063	0.550	0.128	0.600	0.402	0.000	1.432	0.000	2.078	0.000
IVFPQ	0.004	0.980	0.005	0.530	0.012	0.590	0.036	0.510	0.104	0.560	0.339	0.210	0.387	0.000	0.404	0.000
HNSW	0.002	0.810	0.001	0.490	0.005	0.100	0.005	0.000	0.008	0.000	0.021	0.000	0.067	0.000	0.090	0.000
LSH	0.056	0.740	0.043	0.400	0.049	0.400	0.056	0.360	0.130	0.400	0.431	0.000	1.538	0.000	2.227	0.000
glove-25-angular ($n = 1,183,514$)																
	$i = 10$		$i = 144 (n/2^{13})$		$i = 2311 (n/2^9)$		$i = 9246 (n/2^7)$		$i = 36984 (n/2^5)$		$i = 147939 (n/2^3)$		$i = 591757 (n/2^1)$		$i = 887635 (3n/4)$	
EpsHier* (Ours)	0.176	1.000	0.170	1.000	0.168	1.000	0.169	1.000	0.159	1.000	0.021	1.000	0.013	1.000	0.016	1.000
Quick Select	3.470	1.000	3.360	1.000	3.371	1.000	3.382	1.000	3.391	1.000	3.486	1.000	3.574	1.000	3.573	1.000
IVFFlat	0.003	1.000	0.003	1.000	0.007	1.000	0.021	0.980	0.080	0.960	0.327	0.000	1.227	0.000	1.299	0.000
PQ	0.013	0.990	0.014	0.710	0.021	0.470	0.040	0.570	0.097	0.450	0.373	0.000	1.393	0.000	2.032	0.000
IVFPQ	0.003	1.000	0.003	0.940	0.010	0.680	0.034	0.800	0.103	0.640	0.330	0.060	0.383	0.000	0.402	0.000
HNSW	0.002	0.960	0.001	0.710	0.003	0.160	0.004	0.030	0.007	0.000	0.021	0.000	0.068	0.000	0.093	0.000
LSH	0.150	0.880	0.147	0.520	0.138	0.440	0.131	0.540	0.196	0.470	0.472	0.000	1.544	0.000	2.213	0.000
sift-128-euclidean ($n = 1,000,000$)																
	$i = 10$		$i = 122 (n/2^{13})$		$i = 1953 (n/2^9)$		$i = 7812 (n/2^7)$		$i = 31250 (n/2^5)$		$i = 125000 (n/2^3)$		$i = 500000 (n/2^1)$		$i = 750000 (3n/4)$	
EpsHier* (Ours)	0.220	1.000	0.222	1.000	0.228	1.000	0.233	1.000	0.236	1.000	0.220	0.990	0.026	0.990	0.204	1.000
Quick Select	1.707	1.000	1.633	1.000	1.630	1.000	1.638	1.000	1.681	1.000	1.757	1.000	1.815	1.000	1.835	1.000
IVFFlat	0.006	1.000	0.007	1.000	0.011	1.000	0.022	1.000	0.069	0.610	0.250	0.000	0.922	0.000	0.962	0.000
PQ	0.022	1.000	0.022	0.590	0.028	0.480	0.043	0.410	0.086	0.370	0.296	0.000	1.083	0.000	1.603	0.000
IVFPQ	0.003	0.840	0.003	0.410	0.008	0.430	0.027	0.400	0.085	0.530	0.259	0.000	0.287	0.000	0.302	0.000
HNSW	0.001	0.990	0.001	0.760	0.003	0.110	0.004	0.040	0.006	0.000	0.017	0.000	0.056	0.000	0.081	0.000
LSH	0.136	0.700	0.134	0.430	0.123	0.430	0.121	0.430	0.161	0.520	0.370	0.000	1.189	0.000	1.709	0.000
gist-960-euclidean ($n = 1,000,000$)																
	$i = 10$		$i = 122 (n/2^{13})$		$i = 1953 (n/2^9)$		$i = 7812 (n/2^7)$		$i = 31250 (n/2^5)$		$i = 125000 (n/2^3)$		$i = 500000 (n/2^1)$		$i = 750000 (3n/4)$	
EpsHier* (Ours)	0.497	1.000	0.492	1.000	0.496	1.000	0.497	1.000	0.458	1.000	0.388	1.000	0.256	1.000	0.363	1.000
Quick Select	2.131	1.000	2.019	1.000	2.015	1.000	2.022	1.000	2.049	1.000	2.126	1.000	2.238	1.000	2.266	1.000
IVFFlat	0.056	1.000	0.057	1.000	0.062	1.000	0.078	0.990	0.144	0.990	0.386	0.000	1.256	0.000	1.510	0.000
PQ	0.035	0.430	0.035	0.290	0.042	0.280	0.056	0.300	0.118	0.440	0.375	0.000	1.357	0.000	2.003	0.000
IVFPQ	0.005	0.420	0.005	0.200	0.012	0.260	0.033	0.340	0.097	0.380	0.320	0.060	0.451	0.000	0.466	0.000
HNSW	0.002	0.770	0.001	0.490	0.006	0.090	0.006	0.030	0.009	0.000	0.019	0.000	0.059	0.000	0.079	0.000
LSH	0.125	0.160	0.116	0.150	0.109	0.110	0.127	0.150	0.177	0.120	0.429	0.100	1.462	0.000	2.123	0.000

Table 2: Query time and recall at different i values for various datasets. All values shown with 3 decimal precision. Recall values greater than 0.9 are shown in bold with green highlight, and values less than 0.2 are shown with red highlight. The fastest query time among methods with recall > 0.9 is shown in bold for each i .